

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan platform digital seperti media sosial, situs ulasan, dan portal berita telah menyebabkan peningkatan yang sangat besar pada data teks tidak terstruktur yang dihasilkan setiap harinya. Setiap hari, jutaan pengguna menghasilkan teks yang berisi opini, emosi, dan pandangan mereka terhadap berbagai isu, sehingga membentuk kumpulan data yang sangat besar dan kaya informasi (Andreotta, Kirkby, dan Rizzi, 2019). Kondisi ini membuka peluang besar untuk menganalisis opini publik dan tren sosial, tetapi di sisi lain juga menimbulkan tantangan karena proses analisis manual menjadi tidak lagi efektif. Sebagian besar data yang tersedia di internet bersifat tidak terstruktur, sehingga membutuhkan pendekatan komputasional seperti *text mining* dan *Natural Language Processing* (NLP) agar data tersebut dapat diproses dan dimanfaatkan dengan baik (Nikhil, 2015).

Salah satu bidang utama dalam NLP yang terus berkembang adalah analisis sentimen. Analisis ini bertujuan untuk mengidentifikasi serta mengelompokkan opini dan emosi yang terkandung dalam teks (Pang dan L. Lee, 2008). Dalam konteks *machine learning*, analisis sentimen termasuk dalam kategori *supervised learning*, khususnya masalah klasifikasi, sehingga membutuhkan data yang sudah memiliki label sentimen untuk proses pelatihannya (Murfi, Wahyunggoro, dan Sumpeno, 2018). Salah satu pendekatan yang paling umum digunakan adalah *document-level sentiment analysis*, yaitu pendekatan yang mengasumsikan bahwa satu dokumen hanya memiliki satu sentimen dominan secara keseluruhan, seperti positif, negatif, atau netral (Pang dan L. Lee, 2008). Metode ini banyak digunakan karena lebih sederhana dalam pemodelan dan lebih efisien dalam proses anotasi data, karena setiap dokumen hanya memerlukan satu label sentimen. Hal ini membuat pendekatan tersebut sering diterapkan dalam analisis opini berskala besar, seperti pada dataset IMDB dan Yelp (L. Zhang, Shuai Wang, dan B. Liu, 2018; Maas *et al.*, 2011a; X. Zhang, J. Zhao, dan LeCun, 2015a).

Namun demikian, pendekatan *document-level sentiment analysis* memiliki keterbatasan karena menganggap bahwa seluruh isi dokumen memiliki karakter emosi yang seragam. Dalam praktiknya, satu dokumen sering kali membahas bebe-

rapa aspek atau topik yang berbeda, bahkan dengan polaritas sentimen yang tidak sama. Akibatnya, informasi sentimen yang lebih detail dapat terabaikan (Pang dan L. Lee, 2008). Hal ini kemudian mendorong munculnya pendekatan *Topic-Level Sentiment Analysis* (TLSA), yaitu metode yang berfokus pada pengidentifikasian sentimen terhadap topik tertentu di dalam sebuah dokumen (Titov dan McDonald, 2008). Pendekatan ini memungkinkan analisis yang lebih rinci sehingga lebih berguna untuk berbagai kebutuhan seperti evaluasi produk, layanan, maupun kebijakan publik.

Perkembangan dalam TLSA kemudian menghasilkan berbagai model yang mencoba menggabungkan pembelajaran topik dan sentimen secara bersamaan. Salah satu model awal yang menjadi dasar adalah *Joint Sentiment-Topic* (JST), yang memperluas *Latent Dirichlet Allocation* (LDA) dengan menambahkan dimensi sentimen sehingga kedua aspek tersebut dapat dipelajari dalam satu model probabilistik (Lin dan He, 2009; Blei, 2012). Setelah itu, dikembangkan *Topic Sentiment Latent Dirichlet Allocation* (TSLDA) yang berusaha meningkatkan kemampuan model dalam menangkap hubungan antara topik dan sentimen pada berbagai kondisi data (Nguyen dan Shirai, 2015). Meskipun demikian, model-model ini masih berbasis *bag-of-words*, sehingga belum mampu menangkap hubungan semantik antar kata secara mendalam. Untuk mengatasi hal tersebut, dikembangkan *Topic Sentiment model with Word Embedding* (TSWE) yang menggabungkan representasi *word embedding* agar kesamaan makna antar kata dapat ikut dipertimbangkan dalam proses pemodelan (Fu *et al.*, 2016). Pendekatan ini kemudian diperluas melalui *Embedding-enhanced JST* (ELJST) yang menggabungkan *word embedding* dengan pendekatan berbasis *skip-gram* untuk menghasilkan representasi yang lebih stabil dan koheren (Sengupta *et al.*, 2021).

Walaupun TLSA mampu memberikan hasil analisis yang lebih rinci, penerapannya cenderung lebih kompleks karena membutuhkan proses anotasi yang lebih sulit serta pemodelan yang lebih berat. Sebagai alternatif, pendekatan *document-level sentiment analysis* tetap dikembangkan dengan menambahkan informasi topik ke dalam proses pemodelannya. Dalam konteks ini, terdapat dua pendekatan utama, yaitu *Separate Document-Level Sentiment-Topic Model* (SDST) dan *Joint Document-Level Sentiment-Topic Model* (JDST). Pada pendekatan SDST, proses deteksi topik dan klasifikasi sentimen dilakukan secara terpisah menggunakan model yang berbeda, sehingga keduanya tidak saling memengaruhi secara langsung (Sengupta *et al.*, 2021). Sementara itu, JDST mencoba menggabungkan kedua proses tersebut dalam satu kerangka pelatihan sehingga informasi topik dan

sentimen dapat saling berinteraksi selama proses pembelajaran.

Salah satu implementasi JDST berbasis *deep learning* diperkenalkan oleh Gui et al. (2022) melalui pendekatan *Multi-Task Mutual Learning* (MTL-GUI). Model ini menggunakan representasi bersama yang dibangun dengan *Hierarchical Attention Network* berbasis GRU (Chung et al., 2014). Representasi tersebut digunakan langsung untuk klasifikasi sentimen, sedangkan untuk pemodelan topik, representasi yang sama diproses lebih lanjut menggunakan *Variational Autoencoder* (VAE) sebelum kemudian dikelompokkan menggunakan K-Means. Pendekatan ini mampu meningkatkan performa pada kedua tugas, baik sentimen maupun topik. Namun, proses pembentukan topik masih dilakukan setelah representasi utama diperoleh, sehingga struktur klaster belum benar-benar dipelajari secara langsung selama proses pelatihan berlangsung. Hal ini menyebabkan hubungan antara representasi laten dan struktur topik belum sepenuhnya optimal.

Dalam pembelajaran representasi data, *autoencoder* sering digunakan sebagai metode untuk mereduksi dimensi sambil tetap mempertahankan informasi penting dari data masukan (Hinton dan Salakhutdinov, 2006). Pengembangan lebih lanjut dari metode ini adalah *Deep Embedded Clustering* (DEC), yang menggabungkan proses klasterisasi langsung ke dalam ruang laten autoencoder. Dengan pendekatan ini, representasi data dan proses pengelompokan dapat dipelajari secara bersamaan dalam satu proses pelatihan (Xie, Girshick, dan Farhadi, 2016). Hal ini membuat ruang laten yang dihasilkan tidak hanya padat secara representasi, tetapi juga memiliki struktur yang lebih sesuai untuk proses pengelompokan.

Berdasarkan perkembangan tersebut, penelitian ini mengusulkan model *Joint Document-Level Sentiment-Topic* berbasis *Deep Embedded Clustering* (JDST-DEC). Model ini menggunakan representasi dari BERT yang telah dibekukan (*frozen*) sebagai input utama, kemudian diproyeksikan ke dalam ruang laten melalui *shared encoder*. Berbeda dengan pendekatan sebelumnya yang memisahkan proses klasterisasi dan klasifikasi, model ini menggabungkan *clustering head* berbasis DEC dan *sentiment head* dalam satu proses pelatihan yang berjalan secara bersamaan. Dengan demikian, ruang laten yang dihasilkan dapat secara simultan mendukung klasifikasi sentimen dan pembentukan topik yang lebih terstruktur. Dalam kerangka ini, tugas topik tidak dianggap sebagai output utama, melainkan sebagai *auxiliary task* yang membantu membentuk struktur ruang laten agar lebih terorganisir dan lebih mudah diinterpretasikan (Caruana, 1997).

Model yang diusulkan diuji menggunakan dataset IMDB dan Yelp dengan beberapa variasi jumlah topik ($K = 30, 50, 80$) (Maas et al., 2011a; X. Zhang, J. Zhao,

dan LeCun, 2015a). Evaluasi dilakukan pada dua aspek utama, yaitu klasifikasi sentimen dan kualitas topik. Untuk evaluasi sentimen, digunakan metrik seperti *accuracy*, *precision*, *recall*, dan *F1-score* (Sokolova dan Lapalme, 2009). Sementara itu, kualitas topik diukur menggunakan *topic coherence* berbasis NPMI serta *topic diversity* untuk melihat keberagaman hasil kluster (O’Callaghan *et al.*, 2015; Deng, Ruiz, dan Blei, 2020). Model JDST-DEC kemudian dibandingkan dengan beberapa metode pembanding, yaitu NN-BERT, MTL-GUI, BERTopic-HDBSCAN, dan BERTopic-KMeans (Gui *et al.*, 2022; Grootendorst, 2022).

1.2 Rumusan Permasalahan

Dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP), klasifikasi sentimen dan pemodelan topik merupakan dua tugas utama yang sering dilakukan secara terpisah. Pendekatan tradisional seperti *Latent Dirichlet Allocation* (LDA) berbasis probabilistik memiliki keterbatasan dalam menangkap hubungan semantik yang kompleks, sehingga kurang optimal untuk tugas yang membutuhkan pemahaman konteks yang lebih dalam.

Selain itu, pemisahan antara tugas klasifikasi sentimen dan pemodelan topik menyebabkan hilangnya potensi peningkatan performa yang bisa diperoleh dengan memanfaatkan hubungan alami antar tugas tersebut. Model berbasis *deep learning*, seperti *Bidirectional Encoder Representations from Transformers* (BERT), telah terbukti unggul dalam tugas klasifikasi sentimen, namun pendekatan ini belum sepenuhnya diintegrasikan dengan pemodelan topik dalam satu *framework* yang efisien.

Beberapa permasalahan utama yang ingin dijawab dalam penelitian ini adalah:

1. Bagaimana mengembangkan model berbasis *Fully Neural Network* yang dapat menangani klasifikasi sentimen dan pemodelan topik secara simultan?
2. Bagaimana pengaruh pendekatan *Multi-Task Learning* (MTL) terhadap performa model dibandingkan dengan model *single-task*?
3. Bagaimana penerapan *Autoencoder* dapat membantu mengatasi permasalahan dimensi tinggi dalam data teks?
4. Sejauh mana model yang dikembangkan dapat meningkatkan efisiensi dan akurasi dibandingkan dengan metode sebelumnya, seperti analisis sentimen menggunakan BERTopic dan model sentimen BERT?

Melalui penelitian ini, diharapkan dapat diperoleh solusi yang lebih efisien dan akurat dalam menangani kedua tugas secara simultan dengan memanfaatkan pendekatan pembelajaran mendalam berbasis *Fully Neural Network*.

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi model berbasis *Fully Neural Network* yang mampu melakukan klasifikasi sentimen dan pemodelan topik secara simultan menggunakan pendekatan *Multi-Task Learning* (MTL). Dengan pendekatan ini, model diharapkan dapat memanfaatkan hubungan antara kedua tugas tersebut untuk meningkatkan performa keseluruhan dan efisiensi dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP).

Secara spesifik, penelitian ini memiliki beberapa tujuan utama sebagai berikut:

1. Mengembangkan arsitektur berbasis *Fully Neural Network* yang dapat secara bersamaan melakukan klasifikasi sentimen dan pemodelan topik dalam satu model terpadu. Dengan integrasi ini, model diharapkan mampu menangkap hubungan semantik yang lebih dalam dan menghasilkan representasi teks yang lebih bermakna.
2. Menganalisis pengaruh *Multi-Task Learning* (MTL) terhadap efisiensi dan performa model dalam menangani kedua tugas secara simultan. Evaluasi akan dilakukan untuk memahami bagaimana strategi berbagi parameter antara dua tugas ini dapat meningkatkan akurasi, stabilitas, dan efisiensi model dibandingkan dengan pendekatan *single-task*.
3. Menerapkan *Autoencoder* untuk reduksi dimensi guna meningkatkan representasi fitur dan mengoptimalkan performa model *Fully Neural Network*. *Autoencoder* akan digunakan untuk mengekstraksi informasi laten dari teks secara lebih efektif, mengurangi redundansi fitur, dan membantu model dalam menangkap struktur data yang lebih informatif.

Melalui penelitian ini, diharapkan dapat diperoleh model yang lebih efisien, akurat, dan tangguh dalam menangani klasifikasi sentimen dan pemodelan topik secara simultan. Implementasi *Fully Neural Network* dengan MTL dan *Autoencoder* diharapkan dapat memberikan kontribusi dalam peningkatan teknik pemrosesan bahasa alami, terutama dalam konteks analisis teks skala besar dan kompleks.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis, yang diuraikan sebagai berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan mampu memperluas wawasan dalam bidang *Natural Language Processing* (NLP), terutama dalam hal perancangan model yang menggabungkan analisis sentimen dan pemodelan topik dalam satu arsitektur secara simultan. Penggunaan *Deep Embedded Clustering* (DEC) sebagai komponen pemodelan topik berbasis *unsupervised learning* dalam kerangka *joint model* diharapkan dapat menambah khasanah penelitian mengenai integrasi dua tugas yang berbeda dalam satu sistem pembelajaran. Penelitian ini diharapkan dapat menjadi acuan bagi penelitian berikutnya dalam membangun sistem analisis teks yang lebih terintegrasi dan efisien.

2. Manfaat Praktis

Hasil penelitian ini diharapkan dapat digunakan sebagai alat bantu dalam memahami isi ulasan teks secara lebih menyeluruh, baik dari sisi sentimen maupun topik yang terkandung di dalamnya, dalam satu kali proses inferensi. Sistem yang dikembangkan berpotensi diterapkan pada berbagai domain, seperti pemantauan ulasan produk, evaluasi layanan pelanggan, hingga analisis tren opini publik di platform digital. Pendekatan *joint model* ini juga diharapkan dapat mengurangi beban komputasi dibandingkan menjalankan kedua analisis secara terpisah.

1.5 Ruang Lingkup Penelitian

Batasan dalam penelitian ini ditetapkan sebagai berikut:

1. Penelitian ini menggunakan analisis sentimen biner yang terdiri atas dua kelas, yaitu positif dan negatif.
2. Data yang digunakan merupakan *dataset* IMDB dan Yelp berupa kumpulan teks ulasan yang diperoleh melalui platform Hugging Face, yaitu *dataset* IMDB dan *dataset* Yelp Polarity.
3. Jumlah data yang digunakan terdiri atas 50.000 ulasan pada *dataset* IMDB dan 30.000 ulasan pada *dataset* Yelp. Kedua *dataset* disusun dengan distribusi kelas yang seimbang antara sentimen positif dan negatif.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang menjadi dasar penelitian. Pembahasan dimulai dari konsep *analisis sentimen* beserta tingkatan dan pendekatannya, dilanjutkan dengan *pendeteksian topik* dan berbagai metodenya. Selanjutnya diuraikan dasar-dasar *neural network*, mulai dari arsitektur, fungsi aktivasi, *backpropagation*, hingga metrik evaluasi. Bab ini juga membahas *Bidirectional Encoder Representations from Transformers* (BERT) sebagai model representasi teks berbasis *transformer*, *autoencoder* sebagai komponen kompresi representasi, serta *multi-task learning* sebagai kerangka pelatihan model untuk beberapa tugas sekaligus.

2.1 Analisis Sentimen

Platform daring seperti media sosial, forum diskusi, dan situs ulasan produk memungkinkan pengguna mengekspresikan pendapat secara bebas. Hal ini menghasilkan data opini dalam jumlah besar yang terus bertambah, sehingga muncul kebutuhan akan metode komputasional yang mampu mengolahnya secara otomatis.

Analisis sentimen, yang juga dikenal sebagai *opinion mining*, merupakan bidang dalam pemrosesan bahasa alami yang berfokus pada identifikasi dan ekstraksi opini, sentimen, serta emosi yang terkandung dalam teks (B. Liu, 2012). Secara umum, analisis sentimen mengklasifikasikan opini ke dalam beberapa kategori: positif, negatif, atau netral. Klasifikasi ini dapat dilakukan pada berbagai tingkatan, mulai dari keseluruhan dokumen, per kalimat, hingga aspek tertentu dalam teks. Pada penelitian ini, analisis dilakukan pada tingkat dokumen, artinya setiap ulasan diklasifikasikan ke dalam satu label sentimen. Pendekatan ini sesuai karena dataset yang digunakan, yakni Yelp dan IMDB, masing-masing berisi ulasan dari satu pengguna tentang satu entitas.

Analisis sentimen umumnya dilakukan melalui dua pendekatan (Pang, L. Lee, dan Vaithyanathan, 2002). Pendekatan berbasis leksikon menggunakan kamus sentimen yang berisi kata-kata beserta nilai sentimennya untuk menentukan sentimen suatu teks. Pendekatan berbasis *machine learning* melatih model pada data ulasan berlabel sentimen, seperti ulasan produk atau layanan, agar dapat mempelajari pola bahasa yang berkorelasi dengan sentimen tertentu. Pendekatan ini lebih mampu menangani variasi bahasa, konteks, dan domain yang beragam, terutama pada data

berskala besar. Penelitian ini menggunakan pendekatan berbasis *machine learning* sebagai landasan utama.

2.2 Pendeteksian Topik

Topik merujuk pada tema utama yang terkandung dalam sebuah teks. Pendeteksian topik bertujuan menemukan tema-tema yang tersebar dalam kumpulan dokumen tanpa label yang ditentukan sebelumnya (Blei, 2012). Setiap topik direpresentasikan sebagai distribusi probabilitas atas sekumpulan kata, dan maknanya dapat ditelusuri dari kata-kata yang paling sering muncul di dalamnya. Metode ini termasuk ke dalam *unsupervised learning* karena tidak memerlukan anotasi data.

Salah satu metode pendeteksian topik yang umum digunakan adalah BERTopic (Grootendorst, 2022), yang menggabungkan representasi kontekstual dari model bahasa besar dengan *clustering* untuk menghasilkan topik yang koheren secara semantis.

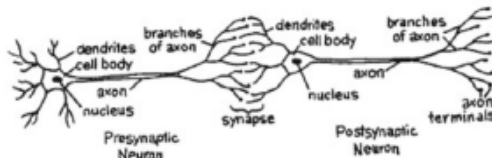
Pada penelitian ini, pendeteksian topik dilakukan dengan pendekatan *clustering*. Dokumen-dokumen yang berdekatan maknanya dikelompokkan ke dalam satu klaster, dan setiap klaster diinterpretasikan sebagai satu topik. Pengelompokan dilakukan pada ruang laten hasil *autoencoder*, sebuah jaringan saraf yang mengompresi teks menjadi representasi yang lebih padat namun tetap mempertahankan informasi semantiknya.

2.3 Neural Network

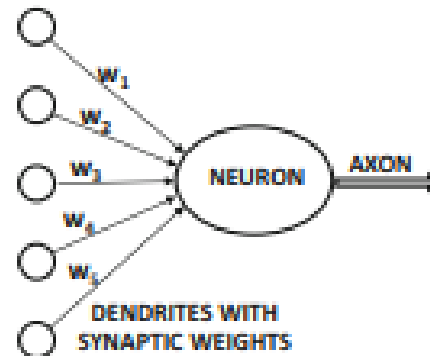
Jaringan saraf tiruan (*Artificial Neural Network/ANN*) adalah model komputasi yang dirancang meniru cara kerja sistem saraf biologis manusia (Aggarwal, 2018). Seperti neuron biologis yang saling terhubung melalui sinapsis, ANN terdiri dari unit-unit komputasi yang saling terhubung melalui bobot yang dapat disesuaikan. Proses pembelajaran berlangsung melalui penyesuaian bobot-bobot tersebut berdasarkan data pelatihan.

Secara umum, arsitektur ANN tersusun atas tiga komponen utama (Silva *et al.*, 2017). Pertama, *input layer*, yaitu lapisan yang menerima data mentah dari luar sistem dan meneruskannya ke lapisan berikutnya tanpa melakukan transformasi. Kedua, *hidden layer*, yaitu lapisan yang menerapkan fungsi aktivasi untuk mengidentifikasi pola kompleks dari data. Ketiga, *output layer*, yaitu lapisan yang menghasilkan prediksi akhir berdasarkan representasi yang telah dibangun oleh lapisan-lapisan sebelumnya.

Gambar 2.1 dan Gambar 2.2 menunjukkan perbandingan antara jaringan saraf biologis dan ANN.



Gambar 2.1: Ilustrasi *neural network* secara biologis (Aggarwal, 2018)



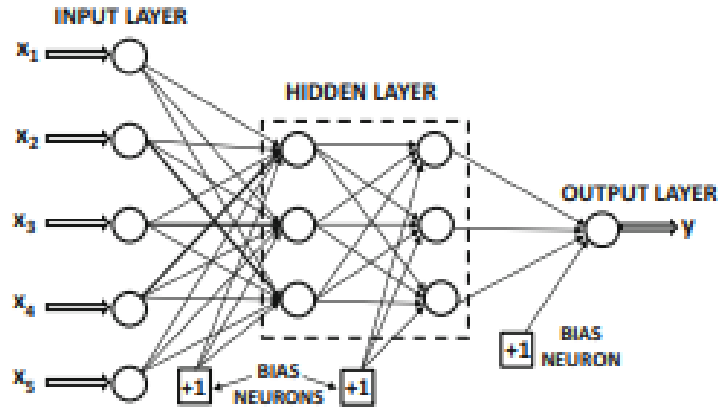
Gambar 2.2: *Artificial Neural Network* dengan representasi bobot sinapsis (Aggarwal, 2018)

Dalam bentuk paling sederhana, sebuah *neural network* hanya memiliki satu lapisan komputasi yang disebut *perceptron*. Pada arsitektur ini, terdapat n neuron *input* yang meneruskan fitur ke satu neuron *output* melalui sekumpulan bobot. Nilai *output* diperoleh dari penjumlahan bobot yang dikalikan dengan masing-masing nilai *input*, ditambah sebuah nilai *bias* b yang berfungsi menggeser ambang batas keputusan model. Secara matematis, prediksi *output* dirumuskan sebagai:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b. \quad (2.1)$$

dengan $\mathbf{w}$ adalah vektor bobot, $\mathbf{x}$ adalah vektor input, dan b adalah skalar bias.

Perceptron hanya mampu memodelkan hubungan linier antara *input* dan *output*, sehingga tidak dapat menangani pola yang lebih kompleks. Untuk mengatasi keterbatasan ini, digunakan *multilayer neural network* yang memiliki satu atau lebih *hidden layer* di antara *input layer* dan *output layer*. Setiap lapisan menerima input dari lapisan sebelumnya dan meneruskannya searah menuju *output layer*. Pola aliran satu arah seperti ini dikenal sebagai *feed-forward*, dan arsitekturnya ditunjukkan pada Gambar 2.3.



Gambar 2.3: Arsitektur *feed-forward neural network* (Aggarwal, 2018)

Misalkan terdapat k *hidden layer* dengan masing-masing $p_1, \dots, p_k$ neuron, $\mathbf{x}$ sebagai vektor input berisi n neuron, $\mathbf{h}_1, \dots, \mathbf{h}_k$ sebagai keluaran tiap *hidden layer* setelah melewati fungsi aktivasi, dan $\mathbf{o}$ sebagai vektor output berisi m neuron. Matriks bobot $\mathbf{W}_1$ berdimensi $n \times p_1$ menghubungkan *input layer* dengan *hidden layer* pertama, $\mathbf{W}_r$ berdimensi $p_{r-1} \times p_r$ menghubungkan antar *hidden layer*, dan $\mathbf{W}_{k+1}$ berdimensi $p_k \times m$ menghubungkan *hidden layer* terakhir ke *output layer*. Proses komputasi pada tiap lapisan dirumuskan sebagai:

$$\mathbf{h}_1 = \phi_1(\mathbf{W}_1^\top \mathbf{x}), \quad (2.2)$$

$$\mathbf{h}_{r+1} = \phi_{r+1}(\mathbf{W}_{r+1}^\top \mathbf{h}_r), \quad r \in \{1, \dots, k-1\}, \quad (2.3)$$

$$\mathbf{o} = \phi_0(\mathbf{W}_{k+1}^\top \mathbf{h}_k). \quad (2.4)$$

dengan ϕ_i adalah fungsi aktivasi yang diterapkan pada lapisan ke- i . Output akhir $\mathbf{o}$ merupakan hasil transformasi bertingkat dari input $\mathbf{x}$ melalui seluruh lapisan jaringan secara berurutan.

2.3.1 Fungsi Aktivasi

Fungsi aktivasi digunakan dalam *neural network* untuk memberikan sifat non-linier pada model. Tanpa fungsi aktivasi, output jaringan hanya merupakan transformasi linier dari input sehingga tidak mampu menangkap pola yang kompleks (Goodfellow, Bengio, dan Courville, 2016). Pada penelitian ini, digunakan tiga fungsi aktivasi utama, yaitu *softmax*, ReLU, dan GELU, yang masing-masing dijelaskan berikut.

2.3.1.1 Softmax

Fungsi *softmax* digunakan untuk mengubah vektor bernilai real menjadi distribusi probabilitas pada K kelas. Melalui proses ini, setiap nilai keluaran berada pada rentang 0 hingga 1, dengan total seluruh probabilitas sama dengan 1. Untuk vektor masukan $\mathbf{v} = (v_1, v_2, \dots, v_K)$, fungsi *softmax* pada elemen ke- j didefinisikan sebagai:

$$\phi(\mathbf{v})_j = \frac{e^{v_j}}{\sum_{k=1}^K e^{v_k}}. \quad (2.5)$$

dengan e adalah bilangan Euler bernilai sekitar 2,718. Elemen dengan nilai v_j yang lebih besar menghasilkan e^{v_j} yang lebih besar, sehingga probabilitas $\phi(\mathbf{v})_j$ yang diperoleh juga lebih tinggi. Fungsi *softmax* umum digunakan pada lapisan keluaran jaringan untuk tugas klasifikasi multikelas karena keluarannya dapat langsung diinterpretasikan sebagai probabilitas prediksi tiap kelas.

2.3.1.2 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) mengatasi masalah *vanishing gradient* dengan cara meneruskan nilai *input* apa adanya jika positif, dan mengembalikan nol jika negatif:

$$\phi(v) = \max\{v, 0\}. \quad (2.6)$$

ReLU lebih efisien secara komputasi dan lebih mudah dilatih pada jaringan yang dalam, karena gradiennya selalu bernilai 1 untuk *input* positif sehingga informasi pembaruan tidak melemah saat merambat mundur melalui banyak lapisan.

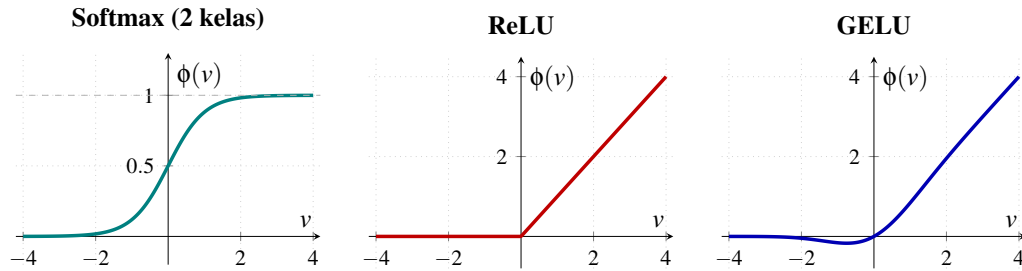
2.3.1.3 Gaussian Error Linear Unit (GELU)

Gaussian Error Linear Unit (GELU) digunakan sebagai fungsi aktivasi pada arsitektur *Transformer* seperti BERT (Hendrycks dan Gimpel, 2016). Berbeda dengan ReLU yang mengubah seluruh nilai negatif menjadi nol, GELU mengatur kontribusi setiap nilai *input* berdasarkan probabilitas kumulatif distribusi Gaussian pada nilai tersebut:

$$\phi(v) = v \cdot \Phi(v) = v \cdot \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{v}{\sqrt{2}}\right) \right]. \quad (2.7)$$

dengan $\Phi(v)$ adalah fungsi distribusi kumulatif Gaussian standar, dan $\operatorname{erf}(\cdot)$ adalah fungsi error dengan rentang nilai $[-1, 1]$, sehingga $\Phi(v) \in [0, 1]$. Untuk v besar dan positif, GELU mendekati perilaku ReLU, sedangkan di sekitar nol transisinya

berlangsung mulus.



Gambar 2.4: Grafik fungsi aktivasi: *Softmax*, ReLU, dan GELU

Perbandingan ketiga fungsi aktivasi ditunjukkan pada Gambar 2.4.

2.3.2 Fungsi Loss

Fungsi *loss* mengukur seberapa jauh prediksi model menyimpang dari nilai yang sebenarnya. Untuk tugas klasifikasi multikelas, *categorical cross-entropy loss* merupakan pilihan yang umum digunakan (Aggarwal, 2018):

$$L = - \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij}). \quad (2.8)$$

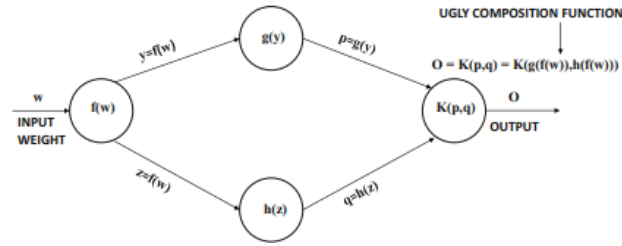
dengan n adalah jumlah sampel, k adalah jumlah kelas, y_{ij} adalah label aktual dalam bentuk *one-hot encoded* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model untuk sampel ke- i pada kelas ke- j .

2.3.3 Backpropagation

Setelah nilai *loss* diperoleh, langkah berikutnya dalam proses pelatihan adalah menghitung gradien terhadap parameter model untuk menentukan arah pembaruan bobot. Proses tersebut dilakukan menggunakan algoritma *backpropagation* (Aggarwal, 2018).

Backpropagation bekerja dengan merambatkan informasi kesalahan dari lapisan keluaran menuju lapisan sebelumnya untuk menghitung seberapa besar kontribusi masing-masing parameter terhadap nilai *loss*. Perhitungan tersebut dilakukan menggunakan aturan rantai (*chain rule*), karena keluaran model umumnya diperoleh melalui serangkaian fungsi yang saling bergantung.

Hubungan tersebut dapat dijelaskan menggunakan graf komputasi pada Gambar 2.5.



Gambar 2.5: Ilustrasi graf komputasi pada proses *backpropagation* (Aggarwal, 2018)

Pada Gambar 2.5, parameter w diproses melalui beberapa tahapan transformasi hingga menghasilkan keluaran akhir o . Keluaran tersebut diperoleh dari dua jalur komputasi yang digabungkan melalui fungsi:

$$o = K(g(f(w)), h(f(w))) \quad (2.9)$$

Persamaan 2.9 menunjukkan bahwa keluaran merupakan fungsi komposit terhadap parameter w . Oleh karena itu, pengaruh perubahan parameter terhadap nilai *loss* dihitung menggunakan aturan rantai sebagai berikut:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \left(\frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial y} \cdot \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial w} \right) \quad (2.10)$$

Pada Persamaan 2.10, $\partial L / \partial o$ menyatakan perubahan nilai *loss* terhadap perubahan keluaran model, sedangkan suku di dalam tanda kurung menyatakan pengaruh parameter terhadap keluaran melalui masing-masing jalur komputasi. Gradien diperoleh dengan mengalikan turunan lokal pada setiap tahapan transformasi dan menjumlahkan kontribusi dari seluruh jalur yang memengaruhi keluaran.

Proses penyebaran gradien dari keluaran menuju parameter pada tahapan komputasi sebelumnya inilah yang disebut sebagai *backpropagation*.

2.3.4 Teknik Optimasi

Proses optimasi merupakan tahapan pencarian parameter yang menghasilkan nilai fungsi *loss* sekecil mungkin selama pelatihan (A. Zhang *et al.*, 2023). Gradien yang diperoleh dari *backpropagation* digunakan untuk memperbarui bobot model secara iteratif.

2.3.4.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) adalah algoritma optimasi yang diperkenalkan oleh (Rumelhart, Hinton, dan Williams, 1986) untuk meminimalkan fungsi *loss* $L(\mathbf{w})$ secara iteratif dengan memperbarui bobot ke arah negatif gradiennya:

$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \eta \nabla L(\mathbf{w})^{[t]}. \quad (2.11)$$

dengan $\mathbf{w}^{[t]}$ adalah vektor bobot pada iterasi ke- t , $\nabla L(\mathbf{w})^{[t]}$ adalah vektor gradien fungsi *loss* terhadap bobot pada iterasi ke- t , dan η adalah skalar *learning rate* yang mengontrol besar langkah pembaruan. Nilai η yang terlalu kecil memperlambat konvergensi, sedangkan nilai yang terlalu besar dapat membuat pelatihan tidak stabil.

2.3.4.2 Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation (Adam), yang dikemukakan oleh (Kingma dan Ba, 2015), mengembangkan SGD dengan mengadaptasi *learning rate* secara individual untuk setiap parameter. Adaptasi ini didasarkan pada dua estimasi momen gradien: momen pertama $\mathbf{v}^{[t]}$ terhadap gradien itu sendiri, dan momen kedua $\mathbf{s}^{[t]}$ terhadap kuadrat gradien:

$$\mathbf{v}^{[t]} = \beta_1 \mathbf{v}^{[t-1]} + (1 - \beta_1) \nabla L(\mathbf{w})^{[t]}. \quad (2.12)$$

$$\mathbf{s}^{[t]} = \beta_2 \mathbf{s}^{[t-1]} + (1 - \beta_2) \left(\nabla L(\mathbf{w})^{[t]} \right)^2. \quad (2.13)$$

dengan β_1 dan β_2 adalah skalar koefisien peluruhan yang mengontrol pengaruh gradien lama terhadap estimasi saat ini. Karena kedua momen diinisialisasi dengan nol, estimasinya cenderung bias ke nol pada iterasi awal. Koreksi bias dilakukan dengan normalisasi:

$$\hat{\mathbf{v}}^{[t]} = \frac{\mathbf{v}^{[t]}}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}^{[t]} = \frac{\mathbf{s}^{[t]}}{1 - \beta_2^t}. \quad (2.14)$$

Seiring bertambahnya t , nilai β_1^t dan β_2^t mendekati nol sehingga koreksi ini semakin tidak berpengaruh. Pembaruan bobot dilakukan menggunakan kedua momen yang telah dikoreksi:

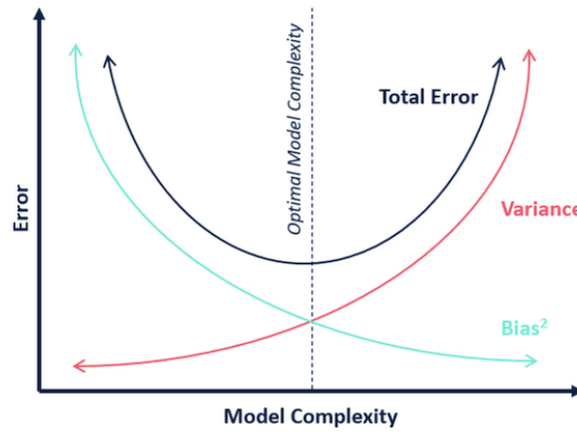
$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}^{[t]} + \epsilon}} \hat{\mathbf{v}}^{[t]}. \quad (2.15)$$

dengan $\epsilon = 10^{-8}$ adalah skalar untuk mencegah pembagian dengan nol (A. Zhang *et al.*, 2023). Pembagian dengan $\sqrt{\hat{\mathbf{s}}^{[t]}}$ memperkecil langkah pembaruan untuk parameter dengan gradien besar dan berfluktuasi, serta memperbesar untuk parameter dengan gradien kecil dan konsisten.

2.3.5 Regularisasi

Dalam melatih model, terdapat dua kondisi yang perlu dihindari, yaitu *underfitting* dan *overfitting*. *Underfitting* terjadi ketika model gagal mempelajari pola data pelatihan, ditandai dengan bias yang tinggi. *Overfitting* terjadi ketika model terlalu menghafalkan data pelatihan sehingga performanya menurun pada data baru, ditandai dengan variansi yang tinggi (Goodfellow, Bengio, dan Courville, 2016).

Gambar 2.6 menunjukkan hubungan antara kompleksitas model dengan bias, variansi, dan total error. Bias menurun seiring meningkatnya kompleksitas model, sementara variansi justru meningkat. Total error yang merupakan kombinasi keduanya mencapai nilai minimum pada titik kompleksitas optimal.



Gambar 2.6: Ilustrasi *bias-variance tradeoff* terhadap kompleksitas model (Aggarwal, 2018, telah diolah kembali)

Untuk mencegah *overfitting*, salah satu teknik yang digunakan adalah *weight decay* atau L_2 regularization, yang menambahkan penalti pada fungsi *loss* proporsional terhadap besar bobot model:

$$\tilde{L}(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2. \quad (2.16)$$

dengan $L(\mathbf{w})$ adalah fungsi *loss* asal, $\|\mathbf{w}\|^2$ adalah jumlah kuadrat seluruh bobot dalam model, dan λ adalah skalar yang mengontrol kekuatan penalti.

Teknik lain yang digunakan adalah *dropout*, yang menonaktifkan neuron secara acak selama pelatihan agar jaringan tidak terlalu bergantung pada neuron tertentu (N. Srivastava *et al.*, 2014). Untuk vektor aktivasi $\mathbf{h} \in \mathbb{R}^p$, setiap neuron ke- i dikalikan dengan variabel acak Bernoulli:

$$m_i \sim \text{Bernoulli}(1 - p_{\text{drop}}), \quad \tilde{h}_i = \frac{m_i}{1 - p_{\text{drop}}} \cdot h_i. \quad (2.17)$$

dengan $p_{\text{drop}} \in [0, 1]$ adalah probabilitas neuron dinonaktifkan, dan faktor $1/(1 - p_{\text{drop}})$ menyeimbangkan skala aktivasi akibat neuron yang dinonaktifkan. Pada tahap evaluasi, *dropout* tidak diterapkan dan seluruh neuron digunakan sepenuhnya ($\tilde{\mathbf{h}} = \mathbf{h}$).

Teknik lain yang juga memberikan efek regularisasi adalah *Batch Normalization* (BatchNorm), yang menormalisasi aktivasi berdasarkan statistik *mini-batch* untuk mempercepat dan menstabilkan pelatihan (Ioffe dan Szegedy, 2015). Untuk aktivasi $\mathcal{B} = \{h_1, \dots, h_m\}$ pada satu *mini-batch*:

$$\hat{h}_i = \frac{h_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \text{BN}(h_i) = \gamma \hat{h}_i + \beta. \quad (2.18)$$

dengan $\mu_{\mathcal{B}}$ dan $\sigma_{\mathcal{B}}^2$ masing-masing adalah rata-rata dan variansi *batch*, ϵ adalah skalar kecil pencegah pembagian dengan nol, serta γ dan β adalah parameter skala dan pergeseran yang dipelajari agar kapasitas representasi jaringan tetap terjaga. Pada tahap evaluasi, $\mu_{\mathcal{B}}$ dan $\sigma_{\mathcal{B}}^2$ digantikan dengan estimasi rata-rata bergerak yang dikumpulkan selama pelatihan.

2.3.6 Metrik Evaluasi

Tahap evaluasi bertujuan untuk mengukur kinerja model pada tugas klasifikasi sentimen maupun klasterisasi topik. Evaluasi klasifikasi dilakukan menggunakan metrik yang diturunkan dari *confusion matrix*, sedangkan evaluasi hasil klasterisasi topik dilakukan menggunakan metrik *topic coherence* dan *topic diversity*. Masing-masing metrik dijelaskan pada subsubbab berikut.

2.3.6.1 Metrik Evaluasi Klasifikasi

Setelah model dilatih, performa klasifikasi dievaluasi menggunakan data yang tidak digunakan selama proses pelatihan. Evaluasi didasarkan pada *confusion matrix*, yaitu tabel yang membandingkan hasil prediksi model dengan label aktual. Dari tabel tersebut diperoleh empat komponen dasar, yaitu *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). TP menunjukkan jumlah sampel positif yang diprediksi dengan benar, TN menunjukkan jumlah sampel negatif yang diprediksi dengan benar, FP menunjukkan jumlah sampel negatif yang salah diprediksi sebagai positif, sedangkan FN menunjukkan jumlah sampel positif yang salah diprediksi sebagai negatif.

Berdasarkan keempat komponen tersebut, digunakan empat metrik evaluasi utama, yaitu akurasi, *precision*, *recall*, dan *F1-Score*.

Akurasi mengukur proporsi keseluruhan prediksi yang benar terhadap seluruh sampel pengujian.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.19)$$

Precision mengukur ketepatan model ketika memprediksi suatu sampel sebagai kelas positif, yaitu proporsi prediksi positif yang benar.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.20)$$

Recall mengukur kemampuan model dalam menemukan seluruh sampel positif yang sebenarnya.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.21)$$

Karena *precision* dan *recall* sering kali saling berkompromi, keduanya dikombinasikan menggunakan *F1-Score*, yaitu rata-rata harmonik antara kedua metrik tersebut.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (2.22)$$

F1-Score memberikan ukuran yang lebih seimbang dibandingkan akurasi, terutama pada dataset yang memiliki distribusi kelas tidak seimbang.

2.3.6.2 Metrik Evaluasi Topik

Kualitas hasil klasterisasi topik dapat dievaluasi menggunakan berbagai metrik kuantitatif. Pada penelitian ini digunakan dua metrik, yaitu *topic coherence* dan *topic diversity*. Kedua metrik dihitung langsung dari dokumen hasil klasterisasi tanpa memerlukan model eksternal.

Topic coherence mengukur sejauh mana kata-kata teratas dalam suatu topik memiliki keterkaitan semantik melalui kemunculan bersama (*co-occurrence*) pada dokumen yang sama. Penelitian ini menggunakan metrik *Normalized Pointwise Mutual Information* (NPMI). Misalkan $\mathcal{D}_k$ adalah himpunan dokumen pada klaster ke- k dan $\{w_1, w_2, \dots, w_N\}$ merupakan N kata teratas pada klaster tersebut setelah proses penghapusan *stop words*. Probabilitas kemunculan setiap kata dan probabilitas kemunculan bersamanya didefinisikan sebagai

$$p(w_i) = \frac{|\{d \in \mathcal{D}_k : w_i \in d\}|}{|\mathcal{D}_k|}, \quad (2.23)$$

$$p(w_j) = \frac{|\{d \in \mathcal{D}_k : w_j \in d\}|}{|\mathcal{D}_k|}, \quad (2.24)$$

$$p(w_i, w_j) = \frac{|\{d \in \mathcal{D}_k : w_i \in d \text{ dan } w_j \in d\}|}{|\mathcal{D}_k|}. \quad (2.25)$$

Nilai *Pointwise Mutual Information* (PMI) dihitung sebagai

$$\text{PMI}(w_i, w_j) = \log \frac{p(w_i, w_j) + \epsilon}{p(w_i)p(w_j) + \epsilon} \quad (2.26)$$

dengan $\epsilon = 10^{-10}$ digunakan untuk menghindari pembagian dengan nol. Selanjutnya nilai PMI dinormalisasi menjadi NPMI.

$$\text{NPMI}(w_i, w_j) = \frac{\text{PMI}(w_i, w_j)}{-\log(p(w_i, w_j) + \epsilon)} \quad (2.27)$$

Nilai NPMI berada pada rentang $[-1, 1]$, di mana nilai yang semakin mendekati 1 menunjukkan bahwa kedua kata semakin sering muncul bersama. Skor *topic coherence* untuk setiap kluster dihitung sebagai rata-rata nilai NPMI seluruh pasangan kata.

$$\text{TC}_k = \frac{1}{\binom{N}{2}} \sum_{j=2}^N \sum_{i=1}^{j-1} \text{NPMI}(w_i, w_j) \quad (2.28)$$

Nilai koherensi keseluruhan diperoleh melalui rata-rata seluruh kluster.

$$\overline{\text{TC}} = \frac{1}{K} \sum_{k=1}^K \text{TC}_k \quad (2.29)$$

Apabila suatu kluster memiliki kurang dari dua kata unik, maka nilai koherensinya ditetapkan sebesar 0.

Topic diversity mengukur tingkat keberagaman kosakata antar topik. Semakin sedikit kata yang muncul berulang pada topik berbeda, semakin tinggi nilai *topic diversity*. Misalkan $\mathcal{W}_k$ adalah himpunan N kata teratas pada kluster ke- k , sedangkan himpunan seluruh kata unik didefinisikan sebagai

$$\mathcal{U} = \bigcup_{k=1}^K \mathcal{W}_k.$$

Nilai *topic diversity* dihitung menggunakan

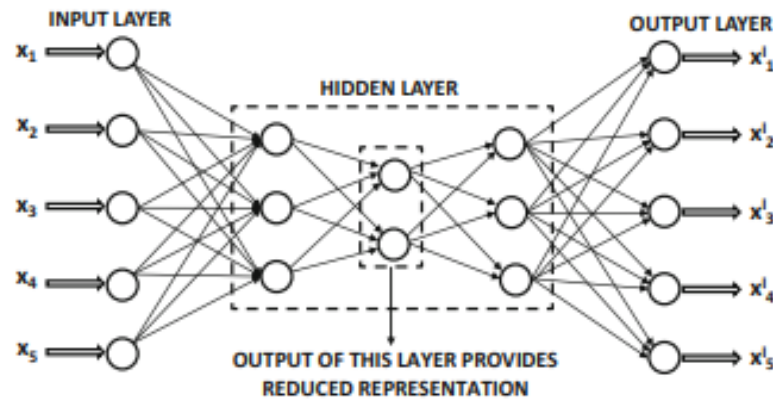
$$TD = \frac{|\mathcal{U}|}{\sum_{k=1}^K |\mathcal{W}_k|} \quad (2.30)$$

Nilai $TD = 1$ menunjukkan bahwa seluruh kata teratas dari setiap topik berbeda satu sama lain, sedangkan nilai yang semakin kecil menunjukkan semakin banyak kosakata yang tumpang tindih antar topik.

2.4 Autoencoder

Pemahaman tentang arsitektur *feed-forward neural network* menjadi landasan untuk memahami *autoencoder*, yang pada dasarnya adalah sebuah jaringan saraf yang dilatih untuk merekonstruksi inputnya (Goodfellow, Bengio, dan Courville, 2016). Karena tidak memerlukan label, *autoencoder* termasuk ke dalam kategori *unsupervised learning* (Hinton dan Salakhutdinov, 2006).

Arsitektur *autoencoder* (Vincent *et al.*, 2010) terdiri dari dua komponen utama: *encoder* yang memetakan representasi input dari dimensi tinggi ke dimensi yang lebih rendah, dan *decoder* yang merekonstruksi representasi tersebut ke dimensi semula. Lapisan tersempit di antara keduanya disebut *bottleneck* atau ruang laten. Arsitektur lengkapnya ditunjukkan pada Gambar 2.7.



Gambar 2.7: Arsitektur *autoencoder* menunjukkan proses *encoding* dan *decoding* (Aggarwal, 2018)

2.4.1 Encoder dan Decoder

Secara formal, misalkan $\mathbf{x} \in \mathbb{R}^D$ adalah vektor input berdimensi D . *Encoder* memetakan $\mathbf{x}$ ke dalam vektor laten $\mathbf{z} \in \mathbb{R}^d$ dengan $d < D$:

$$\mathbf{z} = f_{\theta}(\mathbf{x}) = \sigma_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h). \quad (2.31)$$

dengan $\mathbf{W}_h \in \mathbb{R}^{d \times D}$ adalah matriks bobot *encoder*, $\mathbf{b}_h \in \mathbb{R}^d$ adalah vektor bias, dan $\sigma_h(\cdot)$ adalah fungsi aktivasi. Simbol θ merepresentasikan seluruh parameter pada bagian *encoder*.

Selanjutnya, *decoder* memetakan kembali vektor laten $\mathbf{z}$ ke dalam ruang dimensi asal untuk menghasilkan rekonstruksi $\hat{\mathbf{x}} \in \mathbb{R}^D$:

$$\hat{\mathbf{x}} = g_{\theta'}(\mathbf{z}) = \sigma_o(\mathbf{W}_o \mathbf{z} + \mathbf{b}_o). \quad (2.32)$$

dengan $\mathbf{W}_o \in \mathbb{R}^{D \times d}$ adalah matriks bobot *decoder*, $\mathbf{b}_o \in \mathbb{R}^D$ adalah vektor bias, dan $\sigma_o(\cdot)$ adalah fungsi aktivasi output. Simbol θ' merepresentasikan seluruh parameter pada bagian *decoder*. Secara keseluruhan, *autoencoder* melakukan transformasi:

$$\hat{\mathbf{x}} = g_{\theta'}(f_{\theta}(\mathbf{x})). \quad (2.33)$$

2.4.2 Fungsi Objektif

Autoencoder dilatih dengan meminimalkan selisih antara *input* asal dan hasil rekonstruksinya. Pada penelitian ini, fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE):

$$\mathcal{L}(\theta, \theta') = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - g_{\theta'}(f_{\theta}(\mathbf{x}_i))\|^2. \quad (2.34)$$

dengan n adalah jumlah sampel dan $\|\cdot\|^2$ adalah kuadrat norma Euclidean. Parameter *encoder* θ dan *decoder* θ' diperbarui secara bersama-sama menggunakan *backpropagation* dan optimasi Adam (Vincent *et al.*, 2010).

2.5 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) merupakan model bahasa yang diperkenalkan oleh (Devlin *et al.*, 2018). Berbeda dari model bahasa sebelumnya yang membaca teks secara satu arah, BERT memahami makna suatu kata dengan mempertimbangkan konteks dari kiri maupun kanan secara bersamaan. Kemampuan ini dimungkinkan oleh arsitekturnya yang didasarkan pada *Transformer Encoder*, yaitu mekanisme yang memungkinkan setiap token menghitung relevansinya terhadap token lain dalam kalimat secara paralel.

Sebelum digunakan, BERT dilatih pada korpus teks berskala besar, yaitu Wikipedia bahasa Inggris dan BookCorpus. Pada penelitian ini, bobot BERT yang

sudah terlatih tidak diperbarui selama pelatihan. BERT digunakan murni sebagai ekstraktor fitur yang dibekukan (*frozen*), sehingga representasi yang dihasilkan langsung digunakan sebagai fitur untuk tahap berikutnya.

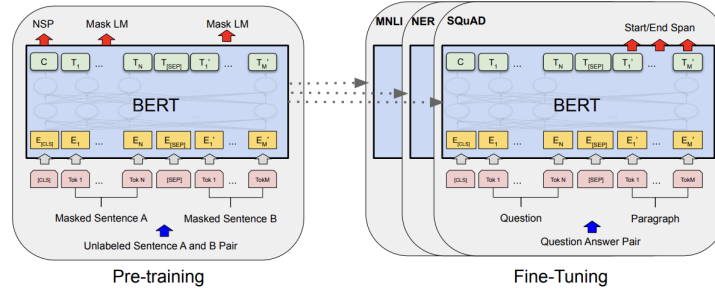
2.5.1 Arsitektur BERT

Arsitektur BERT tersusun dari L lapisan *Transformer Encoder* yang identik, masing-masing terdiri dari dua sub-lapisan: *Multi-Head Self-Attention* dan *Position-wise Feed-Forward Network*. Keluaran setiap lapisan, seperti ditunjukkan pada Gambar 2.8, dihitung sebagai:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell-1)} + \text{MultiHeadAttention}(\mathbf{H}^{(\ell-1)})\right), \quad (2.35)$$

$$\mathbf{H}_{\text{out}}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell)} + \text{FFN}(\mathbf{H}^{(\ell)})\right). \quad (2.36)$$

dengan $\mathbf{H}^{(\ell)}$ adalah matriks *hidden state* pada lapisan ke- ℓ , dan LayerNorm adalah normalisasi lapisan yang membantu stabilitas pelatihan. Jumlah lapisan L , dimensi *hidden state* H , dan jumlah *attention head* A bergantung pada varian model yang dipilih.



Gambar 2.8: Arsitektur BERT dengan beberapa lapisan *Transformer Encoder* (Devlin *et al.*, 2018)

2.5.2 Mekanisme *Self-Attention*

BERT menggunakan mekanisme *self-attention* untuk menentukan seberapa penting suatu kata terhadap kata-kata lain dalam kalimat yang sama. Input $\mathbf{X} \in \mathbb{R}^{n \times d}$ dengan n token dan dimensi d diproyeksikan ke tiga representasi melalui matriks bobot yang dipelajari saat pelatihan, yaitu *query* $\mathbf{Q}$, *key* $\mathbf{K}$, dan *value* $\mathbf{V}$:

$$\mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{n \times d_k}, \quad (2.37)$$

$$\mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{n \times d_k}, \quad (2.38)$$

$$\mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{n \times d_v}. \quad (2.39)$$

dengan $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ dan $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$ adalah matriks bobot yang dapat dipelajari. *Query* merepresentasikan kata yang sedang mencari konteks, *key* merepresentasikan kata-kata yang berpotensi menjadi konteks, dan *value* adalah informasi yang diambil apabila konteks tersebut dianggap relevan.

Skor relevansi antara setiap pasangan kata dihitung melalui perkalian *query* dan *key*, lalu dinormalisasi dengan $\sqrt{d_k}$ untuk menjaga skala nilai. Hasilnya kemudian diubah menjadi distribusi probabilitas menggunakan *softmax*:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.40)$$

Untuk meningkatkan kemampuan representasi, BERT menjalankan mekanisme *attention* ini secara paralel sebanyak h kali pada *subspace* yang berbeda, yang disebut *Multi-Head Attention*. Setiap hasil digabungkan dan diproyeksikan kembali ke dimensi semula:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O. \quad (2.41)$$

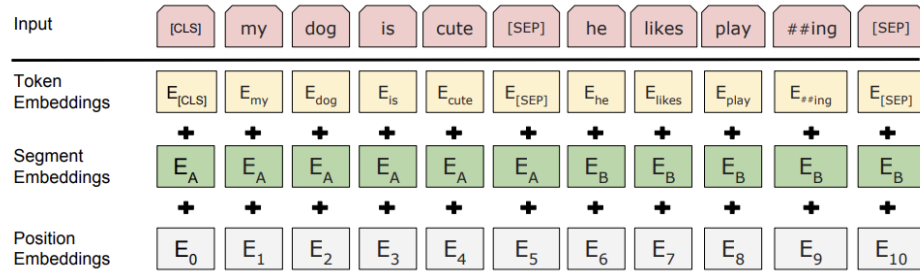
dengan $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q^i, \mathbf{K}\mathbf{W}_K^i, \mathbf{V}\mathbf{W}_V^i)$. Pada BERT-Base, $h = 12$ dan $d_k = d_v = 64$.

2.5.3 Representasi Input BERT

Sebelum teks dapat diproses oleh BERT, setiap token perlu diubah menjadi vektor numerik. BERT merepresentasikan setiap token sebagai penjumlahan dari tiga jenis *embedding* yang masing-masing membawa informasi berbeda:

$$\mathbf{E}_i = \mathbf{E}_{\text{token}}^{(i)} + \mathbf{E}_{\text{segment}}^{(i)} + \mathbf{E}_{\text{position}}^{(i)}. \quad (2.42)$$

Token embedding merepresentasikan makna semantik dari setiap kata atau *sub-word*. BERT menggunakan tokenisasi WordPiece dengan kosakata sebesar 30.000 token, sehingga kata yang tidak dikenal dapat diurai menjadi unit yang lebih kecil. *Segment embedding* menandai asal kalimat dari setiap token, yang berguna ketika *input* terdiri dari dua kalimat. *Position embedding* merepresentasikan posisi setiap token dalam urutan, sebab mekanisme *self-attention* pada dasarnya tidak mengandung informasi urutan. Berbeda dari Transformer asli yang menggunakan rumus sinusoidal, BERT mempelajari *position embedding* langsung dari data dengan panjang urutan maksimum token. Ilustrasi representasi input BERT ditunjukkan pada Gambar 2.9.



Gambar 2.9: Representasi *input* BERT yang terdiri dari *token*, *segment*, dan *position embedding* (Devlin *et al.*, 2018)

Selain token biasa, BERT menyisipkan token khusus [CLS] di awal setiap urutan *input*. Token ini dirancang untuk mengakumulasi informasi dari seluruh token dalam kalimat melalui lapisan-lapisan *self-attention*. Pada penelitian ini, representasi token [CLS] pada lapisan terakhir BERT inilah yang digunakan sebagai vektor fitur untuk setiap ulasan.

Pada penelitian ini, BERT digunakan sepenuhnya sebagai ekstraktor fitur dengan bobot yang dibekukan. Setiap ulasan teks dimasukkan ke dalam BERT dengan panjang maksimum token, kemudian representasi *hidden state* pada posisi token [CLS] dari lapisan terakhir diambil sebagai fitur:

$$\mathbf{f} = \mathbf{H}_{[CLS]}^{(L)} \in \mathbb{R}^d. \quad (2.43)$$

dengan $\mathbf{H}_{[CLS]}^{(L)}$ adalah vektor *hidden state* token [CLS] pada lapisan ke- L , dan d adalah dimensi *hidden state* yang bergantung pada varian model yang digunakan. Vektor $\mathbf{f}$ ini selanjutnya menjadi representasi numerik dari setiap ulasan dan digunakan sebagai masukan *autoencoder* pada tahap berikutnya.

2.6 Multi-Task Learning

Multi-Task Learning (MTL) merupakan pendekatan dalam *machine learning* yang bertujuan meningkatkan kemampuan generalisasi model dengan memanfaatkan informasi dari beberapa tugas yang saling berkaitan secara bersamaan (Caruana, 1997). Ketika dua tugas memiliki keterkaitan, informasi pembelajaran dari satu tugas dapat membantu tugas lainnya karena keduanya berbagi pola atau struktur yang sama dalam data.

Konsep kunci yang mendasari MTL adalah *inductive transfer*, yaitu kemampuan model untuk memindahkan pengetahuan yang dipelajari dari satu tugas ke tugas lainnya. (Caruana, 1997) menjelaskan bahwa MTL mencapai hal ini dengan cara mempelajari beberapa tugas secara paralel menggunakan representasi

bersama. Tugas-tugas lain berfungsi sebagai *inductive bias*, yaitu mekanisme yang mengarahkan model agar mempelajari hipotesis yang dapat menjelaskan lebih dari satu tugas sekaligus (Ruder, 2017).

Pada penelitian ini, MTL digunakan untuk mempelajari analisis sentimen dan pendeteksian topik secara bersamaan. Kedua tugas ini memiliki keterkaitan yang kuat karena sama-sama bergantung pada pemahaman semantis teks ulasan, sehingga representasi yang berguna untuk mendeteksi topik juga cenderung berguna untuk mengklasifikasikan sentimen.

2.6.1 Formulasi MTL

Misalkan terdapat T tugas yang dipelajari secara bersamaan. Untuk setiap tugas $t \in \{1, 2, \dots, T\}$, terdapat dataset $\mathcal{D}_t = \{(\mathbf{x}_i^{(t)}, y_i^{(t)})\}_{i=1}^{n_t}$ dan fungsi *loss* $\mathcal{L}_t$ yang mengukur kesalahan prediksi pada tugas ke- t . Karena MTL mempelajari beberapa tugas secara bersamaan, proses optimasi dilakukan dengan menggabungkan kontribusi seluruh fungsi *loss* ke dalam satu fungsi objektif:

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \alpha_t \mathcal{L}_t(\theta_t, \theta_{\text{shared}}). \quad (2.44)$$

dengan α_t adalah skalar bobot yang mengontrol seberapa besar kontribusi tugas ke- t terhadap total *loss*, θ_t adalah parameter yang spesifik untuk tugas ke- t , dan θ_{shared} adalah parameter yang dibagikan bersama oleh semua tugas. Parameter optimal diperoleh dengan meminimalkan persamaan (2.44) secara bersama-sama:

$$\theta_{\text{shared}}^*, \{\theta_t^*\}_{t=1}^T = \arg \min_{\theta_{\text{shared}}, \{\theta_t\}} \mathcal{L}_{\text{MTL}}. \quad (2.45)$$

Pada penelitian ini, fungsi *loss* total terdiri dari dua komponen. Komponen pertama adalah *clustering loss* berbasis *Kullback-Leibler Divergence* (KLD), yang mengoptimalkan jarak antara distribusi penugasan kluster Q dan distribusi target P . Komponen kedua adalah *sentiment loss* berbasis *categorical cross-entropy* untuk klasifikasi sentimen. Keduanya digabungkan sebagai berikut:

$$\mathcal{L}_{\text{MTL}} = \alpha_1 \cdot \mathcal{L}_{\text{cluster}} + \alpha_2 \cdot \mathcal{L}_{\text{sentiment}}. \quad (2.46)$$

dengan α_1 dan α_2 adalah skalar hiperparameter yang mengontrol bobot masing-masing komponen. *Clustering loss* diformulasikan sebagai KLD antara distribusi target P dan distribusi penugasan Q :

$$\mathcal{L}_{\text{cluster}} = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}. \quad (2.47)$$

dengan q_{ij} adalah probabilitas sampel ke- i termasuk ke kluster ke- j , dan p_{ij} adalah distribusi target yang diturunkan dari Q sebagai berikut:

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} q_{ij'}^2 / \sum_i q_{ij'}}. \quad (2.48)$$

Pembagian dengan $\sum_i q_{ij}$ menormalisasi frekuensi kluster agar kluster yang besar tidak mendominasi, sedangkan operasi kuadrat pada q_{ij} membuat distribusi keanggotaan lebih terkonsentrasi pada satu kluster.

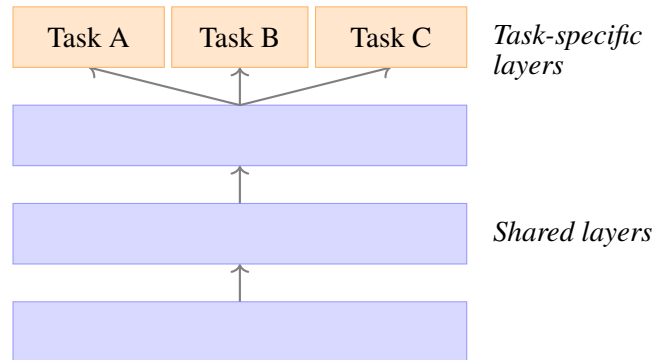
2.6.2 Hard Parameter Sharing

Hard parameter sharing merupakan pendekatan MTL yang paling umum digunakan dalam *deep learning* (Ruder, 2017). Pada pendekatan ini, lapisan tersembunyi bersifat *shared* antar semua tugas, sementara setiap tugas memiliki lapisan *output* tersendiri. Secara formal, untuk input $\mathbf{x}$, proses komputasinya adalah:

$$\mathbf{h}_{\text{shared}} = f_{\text{shared}}(\mathbf{x}; \theta_{\text{shared}}), \quad (2.49)$$

$$\hat{y}_t = f_t(\mathbf{h}_{\text{shared}}; \theta_t), \quad t = 1, \dots, T. \quad (2.50)$$

dengan f_{shared} adalah jaringan bersama yang menghasilkan vektor representasi $\mathbf{h}_{\text{shared}}$, dan f_t adalah jaringan spesifik tugas ke- t . (Baxter, 2000) menunjukkan secara teoritis bahwa mekanisme ini mengurangi risiko *overfitting* karena gradien dari semua tugas mengalir ke lapisan bersama selama *backpropagation*. Arsitektur *hard parameter sharing* diilustrasikan pada Gambar 2.10.



Gambar 2.10: Ilustrasi arsitektur *hard parameter sharing* dalam *Multi-Task Learning* (Ruder, 2017, telah diolah kembali)

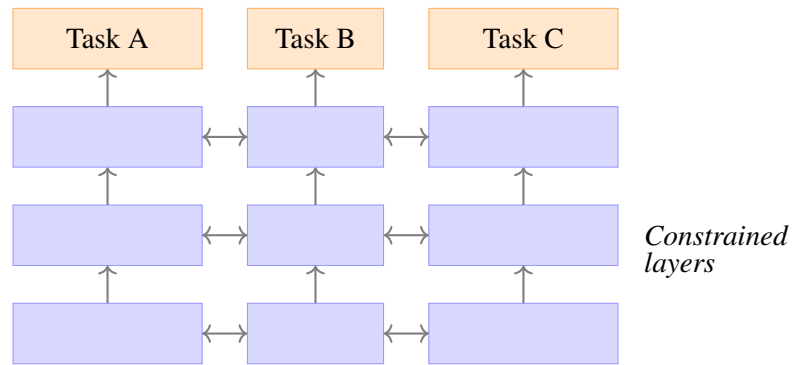
Pada penelitian ini, pendekatan *hard parameter sharing* diterapkan dengan menjadikan *autoencoder* sebagai jaringan bersama. Representasi laten $\mathbf{z}$ yang dihasilkan oleh *encoder* digunakan secara bersamaan oleh dua *head*: satu untuk pendeteksian topik melalui *clustering layer*, dan satu lagi untuk klasifikasi sentimen melalui *sentiment classifier*.

2.6.3 Soft Parameter Sharing

Soft parameter sharing mengambil pendekatan yang berbeda. Setiap tugas memiliki model tersendiri dengan parameter masing-masing, namun parameter antar model diatur agar tidak terlalu jauh berbeda melalui regularisasi (Ruder, 2017). Salah satu cara yang umum digunakan adalah menambahkan penalti ℓ_2 terhadap perbedaan parameter antar tugas:

$$\mathcal{L}_{\text{soft}} = \sum_{t=1}^T \mathcal{L}_t(\theta_t) + \lambda \sum_{t=1}^T \sum_{t'=t+1}^T \|\theta_t - \theta_{t'}\|_2^2. \quad (2.51)$$

dengan λ adalah skalar koefisien regularisasi yang mengontrol seberapa mirip parameter antar tugas harus dijaga, dan $\|\theta_t - \theta_{t'}\|_2^2$ mengukur seberapa jauh kedua set parameter berbeda. Arsitektur *soft parameter sharing* diilustrasikan pada Gambar 2.11.



Gambar 2.11: Ilustrasi arsitektur *soft parameter sharing* (Ruder, 2017, telah diolah kembali)

Perbedaan mendasar antara keduanya terletak pada bagaimana pengetahuan dibagikan. Pada *hard parameter sharing*, bobot benar-benar identik sehingga semua tugas menggunakan representasi yang sama persis. Pada *soft parameter sharing*, setiap tugas tetap memiliki ruang belajar sendiri namun didorong untuk tidak terlalu menyimpang, sehingga lebih fleksibel namun membutuhkan lebih banyak parameter dan pengaturan λ yang tepat.

2.7 Clustering

Clustering merupakan teknik *unsupervised learning* yang mengelompokkan data berdasarkan kemiripan tanpa memerlukan label (Bezdek, 1984). Berdasarkan cara pengelompokannya, terdapat dua pendekatan utama. *Hard clustering* membagi setiap titik data ke satu kluster, sedangkan *soft clustering* memungkinkan satu titik data masuk ke beberapa kluster sekaligus.

Metode *clustering* dapat diklasifikasikan ke dalam beberapa pendekatan, di antaranya *centroid-based*, *distribution-based*, *connectivity-based*, dan *density-based*. Penelitian ini menggunakan pendekatan *centroid-based clustering*, yaitu metode yang memisahkan data berdasarkan jarak setiap titik data terhadap pusat kluster (*centroid*) (Yusdiansyah dan Suryadi, 2019). Untuk mengukur jarak tersebut digunakan jarak Euclidean:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}. \quad (2.52)$$

dengan $\mathbf{p} = (p_1, \dots, p_D)$ dan $\mathbf{q} = (q_1, \dots, q_D)$ adalah dua vektor dalam ruang berdimensi D .

Penelitian ini menggunakan *Deep Embedded Clustering* (DEC) (Xie, Girshick, dan Farhadi, 2016) sebagai metode pendeteksian topik, yang mempelajari representasi laten dan melakukan *clustering* secara bersamaan. DEC menerapkan *soft clustering* melalui distribusi probabilitas penugasan kluster, sehingga setiap ulasan dapat terhubung ke lebih dari satu topik. Sebagai inisialisasi, DEC menggunakan K-Means untuk menentukan pusat kluster awal di ruang *embedding*.

2.7.1 K-Means Clustering

K-Means merupakan algoritma *centroid-based clustering* yang mengelompokkan data dengan meminimalkan total jarak antara setiap titik data dan pusat klusternya, dan termasuk ke dalam kategori *hard clustering* (MacQueen, 1967).

Secara formal, misalkan terdapat himpunan data $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_n \in \mathbb{R}^D$ dan jumlah kluster yang diinginkan adalah K . K-Means meminimalkan fungsi objektif berikut:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2. \quad (2.53)$$

dengan $\boldsymbol{\mu}_k$ adalah vektor *centroid* kluster ke- k , dan r_{nk} adalah skalar variabel indikator biner yang bernilai 1 jika titik data $\mathbf{x}_n$ termasuk kluster ke- k dan 0 jika tidak.

Nilai J mengukur total kuadrat jarak seluruh titik data ke pusat klasternya masing-masing.

Proses optimasi dilakukan secara iteratif melalui dua tahap yang bergantian. Pada tahap pertama (*assignment step*), setiap titik data dialokasikan ke klaster dengan *centroid* terdekat:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{yang lainnya.} \end{cases} \quad (2.54)$$

dengan $\arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2$ menyatakan indeks j yang meminimalkan nilai $\|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2$, yaitu indeks *centroid* terdekat terhadap titik data $\mathbf{x}_n$. Notasi $\arg \max$ yang digunakan pada bagian-bagian selanjutnya memiliki pengertian yang serupa, yaitu mengembalikan indeks yang memaksimalkan suatu fungsi.

Pada tahap kedua (*update step*), *centroid* setiap klaster diperbarui menjadi rata-rata dari seluruh titik yang menjadi anggotanya:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}. \quad (2.55)$$

Pembilang adalah jumlah posisi seluruh anggota klaster ke- k , sedangkan penyebut adalah jumlah anggotanya. Dua tahap ini diulang sampai nilai J tidak berubah signifikan atau jumlah iterasi mencapai batas maksimum. Algoritma lengkapnya dapat dilihat pada Tabel 2.1.

Tabel 2.1: Algoritma K-Means *Clustering*

Algoritma K-Means <i>Clustering</i>	
INPUT: Himpunan data $\{\mathbf{x}_i\}_{i=1}^N$, banyak kluster K , batas toleransi ϵ , dan maksimal iterasi maxiter	
1. Inisialisasi secara acak himpunan <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$	
2. Tetapkan $\text{iter} = 0$	
3. REPEAT	
4. Tetapkan $\text{iter} = \text{iter} + 1$	
5. <i>Assignment step:</i> Cari nilai $r_{nk} = \begin{cases} 1, & k = \arg \min_j \ \mathbf{x}_n - \boldsymbol{\mu}_j\ \\ 0, & \text{yang lainnya} \end{cases}$	
6. <i>Update step:</i> Cari nilai $\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}$	
7. Hitung $J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \ \mathbf{x}_n - \boldsymbol{\mu}_k\ ^2$	
8. UNTIL $J < \epsilon$ OR $\text{iter} > \text{maxiter}$	
9. Untuk setiap $n = 1, 2, \dots, N$, tetapkan nilai $u_n = \arg \max_k (r_{nk})$, dengan $k = 1, 2, \dots, K$	
10. Bentuk $\{C_1, C_2, \dots, C_K\}$, dengan C_k merupakan himpunan anggota data $\mathbf{x}_i$ di mana $u_i = k$	
11. STOP	
OUTPUT: Nilai keanggotaan $\{u_n\}_{n=1}^N$ dan himpunan kluster $\{C_k\}_{k=1}^K$	

2.8 Interpretabilitas Model *Deep Learning*

Interpretabilitas model *deep learning* merujuk pada kemampuan untuk memahami dan menjelaskan keputusan yang dihasilkan oleh model yang bersifat *black-box* (Sundararajan, Taly, dan Yan, 2017). Dalam konteks pemrosesan bahasa alami, interpretabilitas penting karena model perlu mampu menjelaskan mengapa suatu teks diklasifikasikan ke dalam kategori tertentu, terutama pada aplikasi analisis sentimen dan pengelompokan topik.

Secara umum, metode interpretabilitas dapat dikategorikan menjadi dua kelompok (J. Li *et al.*, 2016). Kelompok pertama adalah metode *intrinsic*, yaitu model yang secara bawaan dapat diinterpretasi seperti pohon keputusan dan regresi linier. Kelompok kedua adalah metode *post-hoc*, yaitu metode yang diterapkan setelah model dilatih untuk menganalisis perilaku model yang kompleks tanpa mengubah arsitekturnya.

Dalam penelitian ini, metode *post-hoc* digunakan untuk menganalisis kontribusi setiap token terhadap prediksi sentimen model, yaitu *occlusion-based token attribution* dan *Integrated Gradients*.

2.8.1 Occlusion-based Token Attribution

Occlusion-based attribution merupakan salah satu metode *post-hoc* yang banyak digunakan untuk menginterpretasi model berbasis teks (Zeiler dan Fergus, 2014; J. Li *et al.*, 2016). Metode ini mengukur pentingnya suatu token dengan cara menggantinya dan mengamati perubahan prediksi model.

Pada model berbasis *transformer* seperti BERT, oklusi dilakukan dengan mengganti token target menggunakan token khusus [MASK], sehingga model tetap menerima input dengan panjang yang sama. Pendekatan ini dikenal sebagai *leave-one-out attribution*.

Secara formal, diberikan teks input $\mathbf{x} = (x_1, x_2, \dots, x_T)$ yang terdiri dari T token, representasi kontekstual diperoleh melalui BERT sebagai:

$$\mathbf{h} = \text{BERT}(\mathbf{x}) \in \mathbb{R}^d. \quad (2.56)$$

dengan $\mathbf{h}$ merupakan vektor representasi [CLS] dari lapisan terakhir BERT. Representasi ini kemudian diteruskan ke lapisan klasifikasi sentimen untuk menghasilkan distribusi probabilitas:

$$f(\mathbf{x}) = \text{softmax}(\mathbf{W} \cdot \mathbf{h} + \mathbf{b}) \in \mathbb{R}^C. \quad (2.57)$$

dengan C adalah jumlah kelas sentimen, $\mathbf{W}$ adalah matriks bobot dan $\mathbf{b}$ adalah vektor bias lapisan klasifikasi. Skor atribusi token x_i terhadap kelas sentimen c didefinisikan sebagai selisih probabilitas prediksi sebelum dan sesudah oklusi:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}). \quad (2.58)$$

dengan $f_c(\mathbf{x})$ adalah skalar probabilitas prediksi kelas c pada teks asli, dan $f_c(\mathbf{x}_{\setminus i})$ adalah skalar probabilitas prediksi setelah token x_i diganti dengan [MASK]. Nilai positif berarti token x_i mendukung prediksi kelas c , nilai negatif berarti token menekannya, dan nilai mendekati nol berarti token tidak berpengaruh signifikan.

Dibandingkan dengan metode berbasis gradien, *occlusion-based attribution* lebih mudah diinterpretasi karena tidak bergantung pada komputasi turunan yang kompleks. Namun, metode ini memiliki kompleksitas waktu $O(T)$ terhadap panjang token sehingga lebih mahal secara komputasi untuk teks yang panjang (J. Li *et al.*, 2016).

2.8.2 Integrated Gradients

Integrated Gradients (IG) merupakan metode atribusi berbasis gradien yang diusulkan oleh (Sundararajan, Taly, dan Yan, 2017). Berbeda dengan *occlusion-based attribution* yang bersifat diskrit, IG menghitung kontribusi setiap dimensi input secara kontinu dengan mengintegrasikan gradien prediksi model sepanjang jalur lurus dari *baseline* menuju input asli.

Secara formal, diberikan vektor input $\mathbf{x} \in \mathbb{R}^d$ dan vektor *baseline* $\mathbf{x}' \in \mathbb{R}^d$ yang umumnya berupa vektor nol, atribusi IG untuk dimensi ke- i didefinisikan sebagai:

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha. \quad (2.59)$$

dengan α adalah skalar parameter yang menunjukkan posisi pada jalur dari *baseline* menuju input asli, dan $\frac{\partial f}{\partial x_i}$ adalah skalar gradien prediksi terhadap dimensi ke- i . Dalam praktiknya, integral tersebut diaproksimasi menggunakan *Riemann sum* dengan m langkah interpolasi:

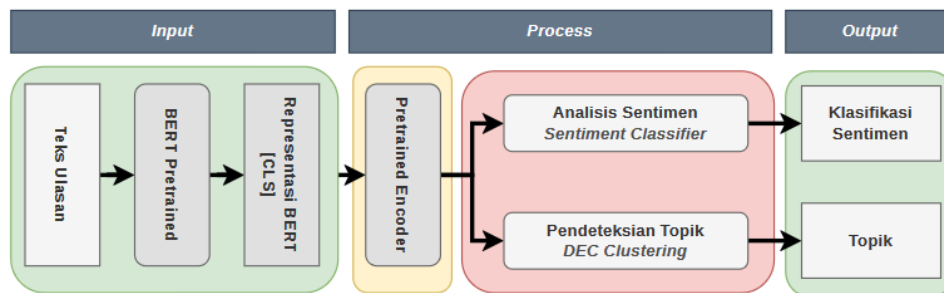
$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f\left(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}')\right)}{\partial x_i} \times \frac{1}{m}. \quad (2.60)$$

Metode IG memenuhi dua karakteristik penting, yaitu *sensitivity* dan *implementation invariance* (Sundararajan, Taly, dan Yan, 2017). *Sensitivity* menjamin bahwa apabila suatu fitur mempengaruhi prediksi model, fitur tersebut akan mendapatkan atribusi bernilai tidak nol. *Implementation invariance* menjamin bahwa dua model yang secara fungsional ekuivalen akan menghasilkan atribusi yang sama meskipun memiliki implementasi internal yang berbeda.

BAB 3

METODE MODEL *JOINT DOCUMENT-LEVEL SENTIMENT-TOPIC* (JDST) BERBASIS *DEEP EMBEDDED CLUSTERING* UNTUK ANALISIS SENTIMEN

Bab ini menjelaskan metode yang digunakan dalam pengembangan model *Joint Document-Level Sentiment-Topic* (JDST-DEC) berbasis *multitask learning*. Pembahasan diawali dengan representasi teks menggunakan BERT pada Subbab 3.1 sebagai pembentuk representasi dokumen. Selanjutnya, Subbab 3.2 membahas proses *pretraining autoencoder* untuk memperoleh *shared encoder*, sedangkan algoritma *Deep Embedded Clustering* (DEC) dijelaskan pada Subbab 3.3. Integrasi *shared encoder*, *clustering head*, dan *sentiment head* ke dalam model JDST-DEC beserta proses pelatihannya dibahas pada bagian pelatihan *multitask*. Tahap terakhir menjelaskan interpretasi model menggunakan analisis atribusi token pada Subbab 3.5. Alur keseluruhan metode penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1: Alur metode analisis sentimen dan pendeteksian topik

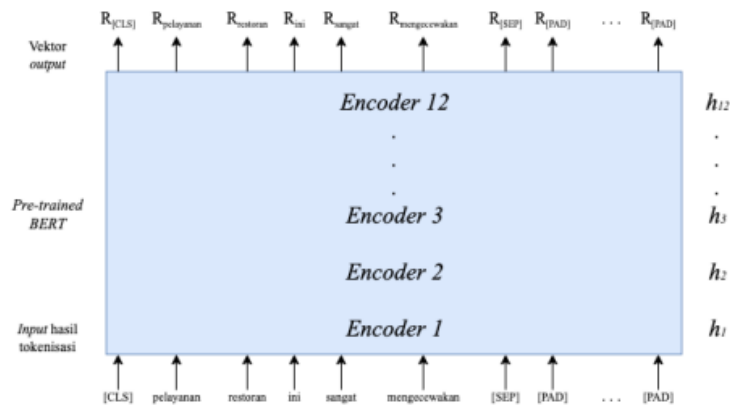
Perbedaan utama JDST-DEC dengan MTL-GUI (Gui *et al.*, 2022) terletak pada mekanisme pembelajaran representasi untuk tugas klasifikasi sentimen dan pendeteksian topik. Pada MTL-GUI, kedua tugas dihubungkan melalui mekanisme *mutual learning* yang meminimalkan divergensi antara *attention vector* pada *bidirectional GRU* berjenjang (klasifikasi sentimen) dan distribusi topik-kata pada *Variational Autoencoder* (VAE) untuk pemodelan topik, sehingga kedua komponen tetap memiliki parameter yang terpisah. Sebaliknya, JDST-DEC menggunakan satu *shared encoder* yang diperbarui secara bersamaan oleh fungsi *loss* klasifikasi sentimen dan *clustering*. Dengan mekanisme tersebut, representasi laten yang dihasilkan digunakan secara bersama oleh kedua tugas sehingga memuat informasi sentimen dan struktur klaster. Selain itu, JDST-DEC memanfaatkan representasi kontekstual

dari BERT yang dibekukan sebagai masukan model, sedangkan MTL-GUI masih menggunakan *embedding* statis dan GRU.

3.1 *Bidirectional Encoder Representations from Transformers* (BERT) Sebagai Model Representasi Teks

Terdapat dua pendekatan utama dalam pemanfaatan model BERT, yaitu *feature-based* dan *fine-tuning*. Pada pendekatan *feature-based*, BERT berfungsi sebagai *feature extractor* tanpa mengubah parameter internalnya. Pada pendekatan *fine-tuning*, lapisan keluaran khusus ditambahkan dan seluruh bobot dilatih ulang secara *end-to-end*.

Pada penelitian ini, BERT digunakan melalui pendekatan *feature-based* dengan bobot yang dibekukan sepenuhnya (*frozen*). Representasi yang dihasilkan kemudian digunakan sebagai *input* untuk *autoencoder* dan pengelompokan topik menggunakan DEC. Contoh representasi teks yang dihasilkan BERT ditunjukkan pada Gambar 3.2.



Gambar 3.2: Representasi teks menggunakan BERT ((Devlin *et al.*, 2018); telah diolah kembali).

Sebagai ilustrasi, misalkan kalimat *input* yang digunakan adalah “*pelayanan restoran ini sangat mengecewakan*”. Proses pembentukan representasi teks oleh BERT dilakukan melalui beberapa tahapan berikut.

1. **Tokenisasi.** Kalimat x dipecah menjadi token menggunakan *WordPiece tokenizer*, kemudian ditambahkan token khusus $[CLS]$ di awal dan $[SEP]$ di akhir:

$$\mathbf{t} = \text{WordPiece}(\mathbf{x}) = [[CLS], t_1, t_2, \dots, t_n, [SEP]] \quad (3.1)$$

Untuk kalimat contoh, hasil tokenisasi berupa:

`[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP]].`

2. **Padding.** Barisan token $\mathbf{t}$ disesuaikan panjangnya menjadi panjang tetap L dengan menambahkan token `[PAD]`:

$$\mathbf{t}' = \text{Pad}(\mathbf{t}, L), \quad |\mathbf{t}'| = L \quad (3.2)$$

Misalnya untuk $L = 10$, diperoleh:

`[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP], [PAD], [PAD], [PAD]].`

3. **Attention Mask.** Untuk membedakan token bermakna dengan token *padding*, dibentuk vektor *attention mask* $\mathbf{a} = [a_1, a_2, \dots, a_L]$ dengan:

$$a_i = \begin{cases} 1, & t'_i \neq [\text{PAD}] \\ 0, & t'_i = [\text{PAD}] \end{cases} \quad (3.3)$$

sehingga untuk contoh di atas diperoleh:

$$\mathbf{a} = [1, 1, 1, 1, 1, 1, 1, 0, 0, 0].$$

4. **Embedding.** Setiap token diubah menjadi vektor numerik melalui penjumlahan tiga komponen, yaitu *token embedding*, *segment embedding*, dan *position embedding*.

- *Token embedding* merepresentasikan makna setiap token. Setiap token dipetakan ke indeks dalam kosakata model BERT yang digunakan, kemudian diubah menjadi vektor berdimensi 768 menggunakan matriks $\mathbf{E}_{\text{token}} \in \mathbb{R}^{|\mathcal{V}| \times 768}$, dengan $|\mathcal{V}|$ menyatakan ukuran kosakata model. Sebagai contoh, token `pelayanan` dipetakan ke indeks $v = 2543$:

$$\mathbf{E}_{\text{token}}[2543] = [0,21, -0,54, 0,83, \dots] \in \mathbb{R}^{768}$$

- *Segment embedding* digunakan untuk membedakan asal kalimat suatu token. Karena penelitian ini hanya menggunakan satu kalimat, seluruh token diberi label Segmen A ($s = 0$):

$$\mathbf{E}_{\text{segment}}[0] = [0,01, 0,01, 0,01, \dots] \in \mathbb{R}^{768}$$

- *Position embedding* memberikan informasi posisi token dalam urutan kalimat. Misalnya, token `[CLS]` berada pada posisi $p = 0$, sedangkan token `pelayanan` berada pada posisi $p = 1$:

$$\mathbf{E}_{\text{position}}[1] = [-0,12, 0,34, -0,07, \dots] \in \mathbb{R}^{768}$$

Ketiga komponen tersebut dijumlahkan untuk membentuk representasi akhir token. Untuk token pelayanan dengan $v = 2543$, $s = 0$, dan $p = 1$, diperoleh:

$$\begin{aligned} \mathbf{e}_{\text{pelayanan}} &= \mathbf{E}_{\text{token}}[2543] + \mathbf{E}_{\text{segment}}[0] + \mathbf{E}_{\text{position}}[1] \\ &= [0,21, -0,54, 0,83, \dots] \\ &\quad + [0,01, 0,01, 0,01, \dots] \\ &\quad + [-0,12, 0,34, -0,07, \dots] \\ &= [0,10, -0,19, 0,77, \dots] \in \mathbb{R}^{768} \end{aligned} \quad (3.4)$$

Proses yang sama dilakukan pada seluruh token sehingga diperoleh matriks *embedding* awal $\mathbf{E} \in \mathbb{R}^{10 \times 768}$ sebagai masukan ke lapisan *encoder* BERT.

5. **Encoding.** Matriks $\mathbf{E}$ diproses melalui 12 lapisan *encoder*. Pada setiap lapisan, mekanisme *multi-head self-attention* menghitung bobot perhatian antar token:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

dengan $d_k = 64$ untuk setiap *head* ($768/12 = 64$), lalu diproses oleh *feed-forward network* dengan koneksi residu dan normalisasi:

$$\mathbf{H}^{(l)} = \text{LayerNorm}\left(\mathbf{H}^{(l-1)} + \text{FFN}\left(\text{MultiHead}\left(\mathbf{H}^{(l-1)}\right)\right)\right)$$

Setelah 12 lapisan, diambil vektor token [CLS] dari keluaran lapisan terakhir sebagai representasi dokumen:

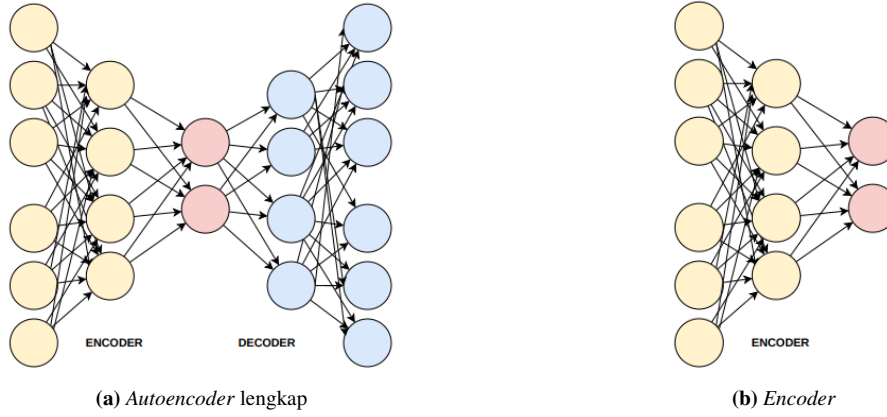
$$\begin{aligned} \mathbf{f} &= \mathbf{H}_{[0,:]}^{(12)} \\ &= [0,31, -0,42, 0,88, \dots] \in \mathbb{R}^{768} \end{aligned} \quad (3.5)$$

Vektor $\mathbf{f}$ ini digunakan sebagai masukan *autoencoder* pada tahap berikutnya, dengan seluruh bobot BERT dibekukan selama pelatihan.

3.2 Shared Encoder

Sebelum digunakan pada pelatihan *multitask*, *autoencoder* terlebih dahulu dilatih secara *unsupervised* melalui proses *pretraining*. Tahap ini bertujuan memperoleh representasi laten yang lebih stabil dibandingkan jika *encoder* langsung dilatih bersama kedua *head* tanpa inisialisasi awal.

Gambar 3.3 menunjukkan ilustrasi arsitektur yang digunakan, mulai dari *autoencoder* lengkap hingga komponen *encoder* yang dipertahankan setelah *pretraining*.



Gambar 3.3: Ilustrasi *autoencoder* lengkap (kiri) dan komponen *encoder* setelah *decoder* dilepas (kanan).

Autoencoder terdiri atas *encoder* f_θ dan *decoder* g_ϕ . *Encoder* berfungsi memetakan representasi *input* $\mathbf{f}$ ke representasi laten $\mathbf{z}$ melalui beberapa lapisan *linear* dengan fungsi aktivasi ReLU:

$$\mathbf{z} = f_\theta(\mathbf{f}) \quad (3.6)$$

Decoder memiliki struktur yang simetris dengan *encoder* dan bertugas mengembalikan $\mathbf{z}$ ke bentuk representasi awal:

$$\hat{\mathbf{f}} = g_\phi(\mathbf{z}) \quad (3.7)$$

sehingga proses rekonstruksi secara keseluruhan dapat dituliskan sebagai:

$$\hat{\mathbf{f}} = g_\phi(f_\theta(\mathbf{f})) \quad (3.8)$$

Pada tahap *pretraining*, parameter θ dan ϕ dioptimalkan bersama dengan meminimalkan selisih antara representasi *input* asli dan hasil rekonstruksinya, menggunakan fungsi *loss Mean Squared Error* (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_i - \hat{\mathbf{f}}_i\|^2 \quad (3.9)$$

dengan N adalah jumlah sampel, $\mathbf{f}_i$ adalah vektor representasi BERT ke- i , dan $\hat{\mathbf{f}}_i$ adalah hasil rekonstruksinya. Fungsi aktivasi ReLU digunakan pada seluruh lapisan tersembunyi karena sederhana secara komputasi dan membantu mengurangi

masalah *vanishing gradient*.

Setelah *pretraining* selesai, *decoder* g_ϕ tidak lagi digunakan, sedangkan parameter *encoder* θ dipertahankan sebagai inisialisasi awal untuk pelatihan *multitask*. Dengan demikian, hanya fungsi f_θ yang digunakan pada tahap selanjutnya:

$$\mathbf{z} = f_\theta(\mathbf{f}), \quad \mathbf{z} \in \mathbb{R}^{256} \quad (3.10)$$

dengan dimensi representasi laten akhir yang dihasilkan *encoder* ditetapkan sebesar 256. Representasi laten $\mathbf{z}$ pada persamaan tersebut digunakan bersama oleh *clustering head* dan *sentiment head* pada tahap pelatihan *multitask*, sehingga *encoder* ini disebut sebagai *shared encoder*.

3.3 Deep Embedded Clustering (DEC)

DEC bertujuan memperbarui representasi laten dan menyesuaikan posisi *centroid* agar sesuai dengan struktur distribusi data pada ruang laten (Xie, Girshick, dan Farhadi, 2016). Berbeda dengan *hard assignment* yang menetapkan setiap sampel ke satu kluster, DEC menggunakan distribusi probabilistik sehingga setiap sampel memiliki derajat keanggotaan terhadap seluruh kluster.

3.3.1 Inisialisasi *Centroid*

Proses DEC diawali dengan menentukan posisi awal *centroid* $\{\mu_j\}_{j=1}^K$ menggunakan algoritma K-Means terhadap representasi laten $\{\mathbf{z}_i\}_{i=1}^N$ hasil *pretraining autoencoder*:

$$\{\mu_j\}_{j=1}^K = \text{KMeans}(\{\mathbf{z}_i\}_{i=1}^N, K) \quad (3.11)$$

dengan K adalah jumlah kluster. Posisi *centroid* yang dihasilkan pada tahap ini menjadi titik awal sebelum diperbarui secara iteratif melalui tahap optimasi berikutnya.

3.3.2 Perhitungan *Soft Assignment*

Dengan posisi *centroid* awal telah diperoleh, setiap data laten $\mathbf{z}_i$ selanjutnya dipetakan menjadi distribusi probabilitas terhadap seluruh *centroid* melalui fungsi $\text{SoftAssign}(\cdot)$, menggunakan distribusi Student's t :

$$q_{ij} = \text{SoftAssign}(\mathbf{z}_i, \boldsymbol{\mu}_j) = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|^2\right)^{-1}}{\sum_{j'=1}^K \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|^2\right)^{-1}} \quad (3.12)$$

dengan $q_{ij} \in [0, 1]$ menyatakan probabilitas sampel ke- i termasuk kluster ke- j . Melalui fungsi ini, setiap sampel tidak ditetapkan secara tegas ke satu kluster, melainkan memiliki derajat keanggotaan q_{ij} terhadap seluruh K kluster sekaligus.

3.3.3 Pembentukan Distribusi Target

Distribusi q_{ij} yang dihasilkan pada tahap sebelumnya kemudian diteruskan ke fungsi $\text{Target}(\cdot)$ untuk ditransformasikan menjadi distribusi target p_{ij} yang lebih terkonsentrasi pada kluster dengan tingkat kepercayaan tinggi. Dengan $f_j = \sum_i q_{ij}$ sebagai frekuensi *soft* kluster ke- j :

$$p_{ij} = \text{Target}(q_{ij}) = \frac{q_{ij}^2 / f_j}{\sum_{j'=1}^K q_{ij'}^2 / f_{j'}} \quad (3.13)$$

Pembagian dengan f_j mencegah kluster berukuran besar mendominasi proses optimasi, sedangkan pengkuadratan q_{ij} membuat distribusi target lebih tegas dibandingkan distribusi *soft assignment* asalnya. Distribusi target $P = \{p_{ij}\}$ inilah yang selanjutnya berperan mengarahkan pembaruan parameter pada tahap optimasi.

3.3.4 Optimasi

Dengan distribusi target P dan distribusi *soft assignment* Q telah tersedia, parameter *encoder* θ dan posisi *centroid* $\{\boldsymbol{\mu}_j\}$ diperbarui secara bersamaan melalui fungsi $\text{Optimize}(\cdot)$, yaitu dengan meminimalkan *Kullback-Leibler Divergence* antara P dan Q :

$$(\theta, \{\boldsymbol{\mu}_j\})_{\text{baru}} = \text{Optimize}(P, Q), \quad \mathcal{L}_{\text{KLD}}(P, Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3.14)$$

Karena $\mathbf{z}_i = f_\theta(\mathbf{f}_i)$, pembaruan θ menyebabkan representasi laten ikut berubah, sehingga *encoder* secara bertahap menghasilkan representasi yang semakin sesuai dengan struktur kluster yang terbentuk.

3.3.5 Kriteria Konvergensi

Setelah parameter diperbarui, label kluster hasil iterasi sebelumnya dibandingkan dengan label kluster terbaru, $y_{\text{cluster},i} = \arg \max_j q_{ij}$, untuk mengukur proporsi perubahan label pada iterasi saat ini:

$$\Delta_{\text{label}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_{\text{prev},i} \neq y_{\text{cluster},i}] \quad (3.15)$$

Keempat tahap di atas (SoftAssign $\rightarrow$ Target $\rightarrow$ Optimize $\rightarrow$ pemeriksaan konvergensi) diulang secara iteratif hingga Δ_{label} berada di bawah batas toleransi τ , yang menandakan bahwa struktur kluster pada ruang laten telah stabil.

Keseluruhan tahapan tersebut dirangkum pada algoritma berikut.

Tabel 3.1: Algoritma *Deep Embedded Clustering* (DEC)

Algoritma <i>Deep Embedded Clustering</i> (DEC)	
INPUT	Representasi laten $\{\mathbf{z}_i\}_{i=1}^N \in \mathbb{R}^{256}$ hasil <i>pretraining autoencoder</i> ; jumlah kluster K ; <i>batch size</i> b ; batas toleransi konvergensi τ ; jumlah iterasi maksimum T_{max}
OUTPUT	Posisi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$; label kluster $\{y_{\text{cluster},i}\}$
1.	Inisialisasi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$ menggunakan K-Means pada $\{\mathbf{z}_i\}$
2.	FOR iter = 1 hingga T_{max} 2.1 Simpan label kluster sebelumnya: $y_{\text{prev}} \leftarrow \arg \max_j q_{ij}$ 2.2 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.12) 2.3 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.13) 2.4 Perbarui parameter <i>encoder</i> dan <i>centroid</i> dengan meminimalkan $\mathcal{L}_{\text{KLD}}(P, Q)$ 2.5 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}[y_{\text{prev},i} \neq \arg \max_j q_{ij}]$ 2.6 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Kembalikan <i>centroid</i> $\{\boldsymbol{\mu}_j\}$ dan label kluster $\{y_{\text{cluster},i}\}$

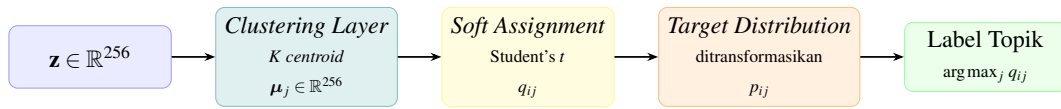
3.4 Pelatihan *Multitask*: JDST-DEC

Pendekatan *multitask* memungkinkan representasi laten dipelajari secara bersama untuk dua tugas, yaitu klasifikasi sentimen dan klusterisasi topik. Melalui pendekatan ini, proses pembelajaran representasi mempertimbangkan informasi sentimen dan struktur topik secara simultan dalam satu ruang laten, berbeda dengan pendekatan yang mengoptimalkan kedua tugas secara terpisah. Kombinasi DEC de-

ngan *sentiment head* dalam kerangka *multitask* ini selanjutnya disebut JDST-DEC (*Joint Document-level Sentiment-Topic*).

3.4.1 Clustering Head

Vektor laten $\mathbf{z}$ yang dihasilkan *encoder* diteruskan ke *clustering head* untuk pendeteksian topik. *Clustering head* mengadopsi mekanisme DEC sehingga setiap sampel memiliki derajat keanggotaan probabilistik terhadap seluruh kluster, bukan penetapan ke satu kluster secara langsung. Gambar 3.4 mengilustrasikan arsitekturnya.

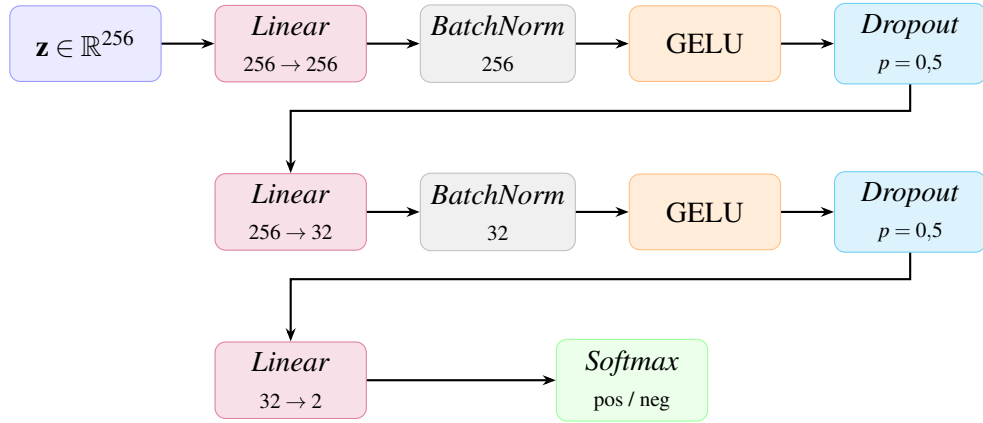


Gambar 3.4: Arsitektur *clustering head* berbasis DEC.

Clustering head terdiri atas satu lapisan kluster yang menyimpan K centroid $\mu_j \in \mathbb{R}^{256}$. Untuk setiap vektor laten $\mathbf{z}_i$, lapisan ini menghitung distribusi *soft assignment* q_{ij} terhadap seluruh *centroid* menggunakan distribusi Student's t sebagaimana didefinisikan pada Persamaan 3.12. Distribusi tersebut kemudian ditransformasikan menjadi distribusi target p_{ij} (Persamaan 3.13), dan label topik akhir diperoleh melalui $\arg \max_j q_{ij}$.

3.4.2 Analisis Sentimen Menggunakan *Sentiment Head*

Vektor laten $\mathbf{z} \in \mathbb{R}^{256}$ yang dihasilkan *encoder* diteruskan ke *sentiment head* untuk diklasifikasikan ke salah satu dari dua kelas sentimen. Arsitektur *sentiment head* terdiri atas tiga lapisan linear bertingkat dengan dimensi $256 \rightarrow 256 \rightarrow 32 \rightarrow 2$. Setiap lapisan tersembunyi dilengkapi dengan *batch normalization*, fungsi aktivasi *Gaussian Error Linear Unit* (GELU), dan *dropout* dengan probabilitas $p = 0,5$. Lapisan keluaran berdimensi dua diikuti fungsi *softmax* untuk menghasilkan distribusi probabilitas antarkelas. Gambar 3.5 mengilustrasikan arsitektur lengkapnya.



Gambar 3.5: Arsitektur *sentiment head*.

Secara formal, proses klasifikasi pada *sentiment head* dituliskan sebagai komposisi tiga lapisan linear bertingkat. Lapisan tersembunyi pertama memetakan $\mathbf{z} \in \mathbb{R}^{256}$ menjadi $\mathbf{h}_1 \in \mathbb{R}^{256}$:

$$\mathbf{h}_1 = \text{Dropout}\left(\text{GELU}\left(\text{BN}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1)\right)\right), \quad \mathbf{W}_1 \in \mathbb{R}^{256 \times 256} \quad (3.16)$$

Lapisan tersembunyi kedua memetakan $\mathbf{h}_1$ menjadi $\mathbf{h}_2 \in \mathbb{R}^{32}$:

$$\mathbf{h}_2 = \text{Dropout}\left(\text{GELU}\left(\text{BN}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)\right)\right), \quad \mathbf{W}_2 \in \mathbb{R}^{32 \times 256} \quad (3.17)$$

Lapisan keluaran memetakan $\mathbf{h}_2$ menjadi distribusi probabilitas terhadap dua kelas sentimen:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c\mathbf{h}_2 + \mathbf{b}_c), \quad \mathbf{W}_c \in \mathbb{R}^{2 \times 32} \quad (3.18)$$

dengan $\mathbf{W}_c$ dan $\mathbf{b}_c$ masing-masing adalah bobot dan bias pada lapisan keluaran. Fungsi *softmax* mengubah nilai logit menjadi distribusi probabilitas, di mana setiap elemen $\hat{y}_i \in [0, 1]$ dan $\sum_i \hat{y}_i = 1$.

Selama pelatihan, prediksi $\hat{\mathbf{y}}$ dibandingkan dengan label sebenarnya $\mathbf{y}$ menggunakan fungsi *Categorical Cross-Entropy loss*:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^N \sum_{j=1}^2 y_{ij} \log \hat{y}_{ij} \quad (3.19)$$

dengan N adalah jumlah sampel, y_{ij} adalah label aktual dalam bentuk *one-hot* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model.

3.4.3 Pelatihan *Joint* dengan *Multitask Loss*

Pada tahap pelatihan *joint*, model dioptimalkan menggunakan kombinasi berbobot antara *clustering loss* dan *sentiment loss*. Fungsi objektif total dirumuskan sebagai:

$$\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) \quad (3.20)$$

dengan $\mathcal{L}_{\text{KLD}}$ digunakan untuk mengoptimalkan kualitas struktur klaster dan $\mathcal{L}_{\text{CE}}$ digunakan untuk meningkatkan kemampuan prediksi sentimen. Parameter γ dan η mengontrol kontribusi relatif masing-masing objektif dan ditentukan melalui pencarian *hyperparameter* pada data validasi. Nilai γ yang lebih besar akan lebih menekankan pembentukan struktur klaster, sedangkan nilai η yang lebih besar akan lebih mengarahkan representasi laten terhadap pemisahan kelas sentimen.

Pada proses optimasi, setiap komponen model menerima sinyal gradien yang berbeda sesuai perannya. Pada Persamaan 3.21–3.23, λ menyatakan *learning rate* dan m menyatakan jumlah sampel dalam satu *mini-batch*.

$$\mu_j = \mu_j - \frac{\lambda}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}_{\text{KLD}}}{\partial \mu_j} \quad (3.21)$$

Sementara itu, parameter pada *sentiment head* ($\mathbf{W}_c, \mathbf{b}_c$) hanya diperbarui berdasarkan *sentiment loss*:

$$\mathbf{W}_c = \mathbf{W}_c - \frac{\lambda}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}_{\text{CE}}}{\partial \mathbf{W}_c} \quad (3.22)$$

Subbab Berbeda dengan kedua *head*, parameter *encoder* θ menerima gabungan gradien dari kedua fungsi objektif karena representasi laten digunakan secara bersama oleh proses pendeteksian topik dan klasifikasi sentimen. Aturan pembaruan parameter *encoder* dituliskan sebagai:

$$\theta = \theta - \frac{\lambda}{m} \sum_{i=1}^m \left(\gamma \frac{\partial \mathcal{L}_{\text{KLD}}}{\partial \theta} + \eta \frac{\partial \mathcal{L}_{\text{CE}}}{\partial \theta} \right) \quad (3.23)$$

Mekanisme ini serupa dengan IDEC (Guo *et al.*, 2017), namun fungsi rekonstruksi pada IDEC digantikan oleh fungsi *Categorical Cross-Entropy loss* sebagai *auxiliary task* untuk analisis sentimen.

3.4.4 Algoritma JDST-DEC

Tabel ?? menyajikan prosedur lengkap pelatihan JDST-DEC. Algoritma ini mengasumsikan *encoder* telah diinisialisasi melalui *pretraining autoencoder* dan

centroid telah diinisialisasi menggunakan K-Means sebagaimana dijelaskan pada Subbab 3.3.

Tabel 3.2: Algoritma *Deep Embedded Clustering* (DEC)

Algoritma <i>Deep Embedded Clustering</i> (DEC)	
INPUT	Representasi laten $\{\mathbf{z}_i\}_{i=1}^N \in \mathbb{R}^{256}$ hasil <i>pretraining autoencoder</i> ; jumlah kluster K ; <i>batch size</i> b ; batas toleransi konvergensi τ ; jumlah iterasi maksimum $T_{\max}$
OUTPUT	Posisi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$; label kluster $\{y_{\text{cluster},i}\}$
1.	Inisialisasi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$ menggunakan K-Means pada $\{\mathbf{z}_i\}$; hitung $q_{ij}^{(0)}$ menggunakan distribusi Student's t (Persamaan 3.12); tetapkan $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}^{(0)}$
2.	FOR iter = 1 hingga $T_{\max}$ 2.1 Simpan label kluster sebelumnya: $y_{\text{prev}} \leftarrow y_{\text{cluster}}$ 2.2 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.12) 2.3 Perbarui label kluster: $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}$ 2.4 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.13) 2.5 Perbarui parameter <i>encoder</i> dan <i>centroid</i> dengan meminimalkan $\mathcal{L}_{\text{KLD}}(P, Q)$ 2.6 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}[y_{\text{prev},i} \neq y_{\text{cluster},i}]$ 2.7 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Kembalikan <i>centroid</i> $\{\boldsymbol{\mu}_j\}$ dan label kluster $\{y_{\text{cluster},i}\}$

3.5 Interpretasi Model dengan Analisis Atribusi Token

Interpretasi model dilakukan menggunakan metode *post-hoc occlusion-based token attribution* pada level token. Analisis ini diterapkan setelah proses pelatihan selesai menggunakan bobot model terbaik untuk mengidentifikasi kontribusi setiap token terhadap prediksi sentimen serta mengamati pola linguistik pada masing-masing kluster.

Meskipun *Integrated Gradients* dibahas pada tinjauan pustaka sebagai salah satu pendekatan interpretabilitas berbasis gradien, metode tersebut tidak diterapkan pada penelitian ini karena interpretasi difokuskan pada analisis perubahan prediksi akibat manipulasi token secara langsung pada tingkat masukan. Pendekatan ini dipilih karena memungkinkan pengamatan kontribusi masing-masing token terhadap perubahan hasil prediksi secara langsung pada tingkat masukan. Dengan demikian, pengaruh setiap token terhadap keputusan model dapat diinterpretasikan secara

lebih jelas. Oleh karena itu, penelitian ini menggunakan metode *occlusion-based token attribution* sebagai pendekatan interpretabilitas.

Secara formal, probabilitas prediksi terhadap kelas c untuk teks *input* $\mathbf{x}$ didefinisikan sebagai komposisi fungsi dari BERT, *shared encoder*, dan *sentiment head*:

$$f_c(\mathbf{x}) = \text{SentimentHead}_c\left(f_\theta(\text{BERT}(\mathbf{x}))\right) \quad (3.24)$$

Nilai $f_c(\mathbf{x})$ pada Persamaan 3.24 inilah yang berperan sebagai probabilitas *base-line* sebelum dilakukan manipulasi token. Selanjutnya, setiap token x_i dalam teks, kecuali token spesial [CLS], [SEP], dan [PAD], digantikan dengan token [MASK] melalui fungsi $\text{Mask}(\cdot)$ sehingga menghasilkan varian teks:

$$\mathbf{x}_{\setminus i} = \text{Mask}(\mathbf{x}, i) \quad (3.25)$$

Varian $\mathbf{x}_{\setminus i}$ tersebut kemudian diproses kembali melalui fungsi $f_c(\cdot)$ yang sama pada Persamaan 3.24 untuk memperoleh probabilitas prediksi yang diperbarui, yaitu $f_c(\mathbf{x}_{\setminus i})$.

Skor atribusi token x_i terhadap kelas c dihitung sebagai selisih antara probabilitas prediksi sebelum dan sesudah token dimask:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}) \quad (3.26)$$

Nilai atribusi positif menunjukkan bahwa token berkontribusi terhadap prediksi kelas c , sedangkan nilai negatif menunjukkan kontribusi yang berlawanan arah. Nilai yang mendekati nol mengindikasikan bahwa token memberikan pengaruh yang relatif kecil terhadap keputusan model.

Untuk memperoleh interpretasi pada level topik, skor atribusi diagregasi berdasarkan hasil klasterisasi. Misalkan $C_k = \{i : y_{\text{cluster}, i} = k\}$ merupakan himpunan sampel yang termasuk ke dalam klaster ke- k . Rata-rata skor atribusi token w pada klaster k didefinisikan sebagai:

$$\overline{\text{Attr}}(w, k, c) = \frac{1}{|C_k|} \sum_{i \in C_k} \text{Attr}(x_i^{(w)}, c) \quad (3.27)$$

dengan $x_i^{(w)}$ menyatakan posisi token w pada sampel ke- i dan $|C_k|$ menyatakan jumlah sampel dalam klaster k .

Untuk meningkatkan reliabilitas interpretasi, hanya token yang muncul minimal pada dua sampel dalam klaster yang sama yang diikutsertakan dalam proses agregasi:

$$\mathcal{V}_k = \left\{ w \left| \sum_{i \in C_k} \mathbf{1}[w \in \mathbf{x}_i] \geq 2 \right. \right\} \quad (3.28)$$

Token dalam $\mathcal{V}_k$ kemudian diurutkan berdasarkan nilai absolut skor atribusi rata-rata untuk memperoleh daftar sebanyak R token paling berpengaruh pada kluster ke- k :

$$\text{TopTokens}(k, c) = \arg \max_{w \in \mathcal{V}_k} \text{top-}R \left| \overline{\text{Attr}}(w, k, c) \right| \quad (3.29)$$

Dalam implementasinya, kosakata yang dievaluasi dibatasi pada *top-N* kata berdasarkan nilai TF-IDF pada masing-masing kluster, sehingga perhitungan atribusi difokuskan pada kata-kata yang paling merepresentasikan karakteristik tiap kluster.

BAB 4

HASIL DAN SIMULASI DAN PEMBAHASAN

Bab ini membahas hasil simulasi dan analisis kinerja model *Joint Sentiment-Topic* berbasis *Multi-Task Learning* yang diusulkan. Data yang digunakan berasal dari dataset IMDB dan Yelp yang telah melalui tahap prapengolahan dan pembagian data. Kinerja model dievaluasi dari dua aspek, yaitu analisis sentimen dan pendeteksian topik. Pada analisis sentimen, performa diukur menggunakan metrik *accuracy* dan *F1-score*. Pada pendeteksian topik, kualitas topik dinilai menggunakan metrik *Topic Coherence* berbasis NPMI dan *Topic Diversity*. Hasil interpretasi model melalui analisis atribusi token juga dipaparkan untuk menggambarkan karakteristik linguistik tiap kluster topik.

4.1 Spesifikasi Mesin dan Perangkat Lunak

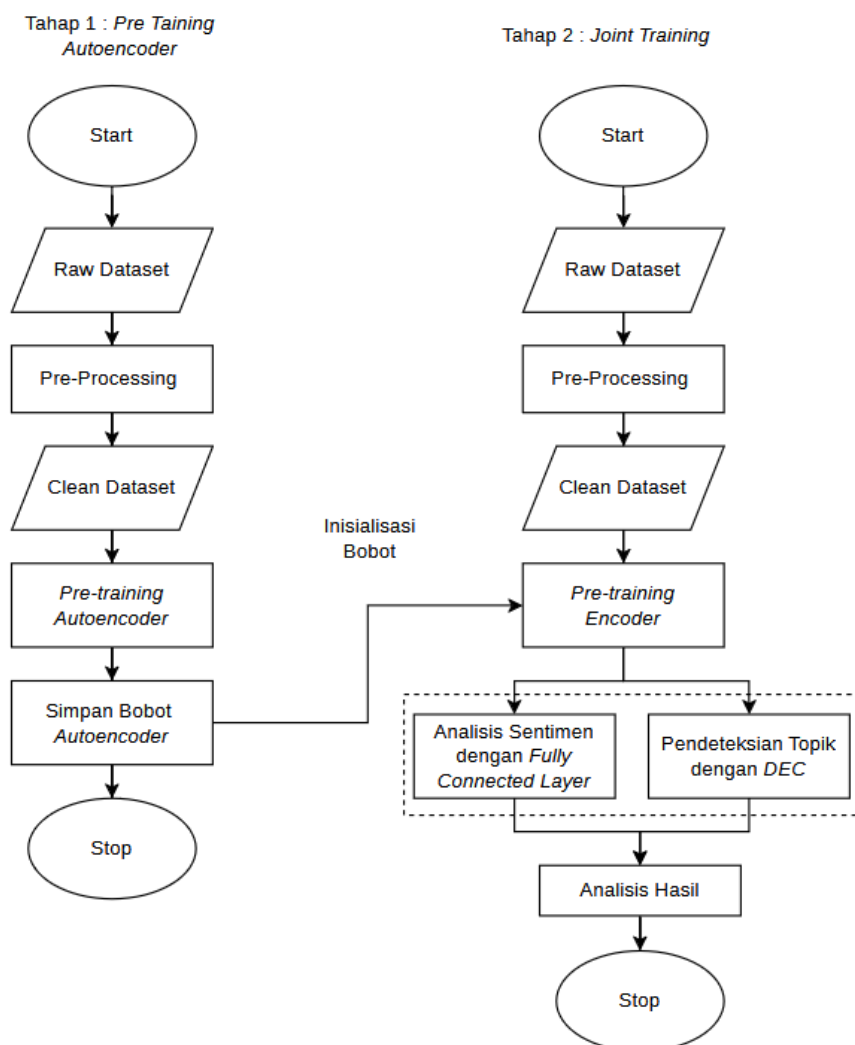
Spesifikasi mesin dan perangkat lunak yang digunakan dalam proses simulasi pada penelitian ini dapat dilihat pada Tabel 4.1.

Tabel 4.1: Spesifikasi mesin dan perangkat lunak

Mesin	
<i>Central Processing Unit</i> (CPU)	Apple M1
Memory	8 GB
<i>Graphic Processing Unit</i> (GPU)	Apple M1 GPU (8-core)
Perangkat Lunak	
Perangkat Lunak Pemrograman	Google Colaboratory Kaggle Notebooks Visual Studio Code
Bahasa Pemrograman	Python 3.13.3
Pustaka yang Digunakan	- os, re, csv, json, glob - numpy, pandas - PyTorch (torch) - scikit-learn, scipy - transformers - datasets - tqdm - seaborn

4.2 Tahapan Umum Simulasi

Tahapan umum simulasi pada penelitian ini digambarkan dalam diagram alur pada Gambar 4.1.



Gambar 4.1: Diagram alur simulasi penelitian.

Proses dimulai dengan pengumpulan data ulasan yang kemudian melewati tahap prapengolahan untuk menghasilkan data yang bersih. Data tersebut selanjutnya diproses oleh BERT yang digunakan secara *feature-based* dengan bobot yang dibekukan sehingga menghasilkan representasi vektor [CLS] berdimensi 768 untuk setiap dokumen.

Pada tahap pertama, representasi BERT digunakan untuk melatih *autoencoder* guna memperoleh representasi laten berdimensi lebih rendah yang mampu mempertahankan informasi penting dari data masukan. Representasi laten hasil pelatihan ini kemudian digunakan sebagai inisialisasi untuk tahap berikutnya.

Pada tahap kedua, representasi laten dimanfaatkan secara bersamaan oleh dua tugas, yaitu analisis sentimen melalui *sentiment head* dan pendeteksian topik melalui *Deep Embedded Clustering* (DEC) dengan mekanisme pelatihan *joint*. Setelah proses pelatihan selesai, kinerja kedua tugas dievaluasi menggunakan

metrik yang sesuai.

4.3 Data

Subbab ini menjelaskan data yang digunakan dalam penelitian, meliputi sumber data, karakteristik dataset, serta tahapan persiapan data sebelum digunakan dalam pelatihan model. Secara umum, proses pengolahan data dimulai dari pengumpulan dataset ulasan, lalu dilanjutkan dengan prapengolahan teks, pembentukan representasi menggunakan BERT, dan pembagian data.

4.3.1 Pengumpulan Data

Penelitian ini menggunakan dua dataset ulasan berbahasa Inggris, yaitu IMDB Movie Reviews dan Yelp Review Dataset. Kedua dataset dipilih karena umum digunakan sebagai *benchmark* pada penelitian analisis sentimen dan pendeteksian topik, serta keduanya memiliki label sentimen biner.

Dataset IMDB Movie Reviews terdiri dari 50.000 ulasan film yang diunggah oleh (Maas *et al.*, 2011b) dan tersedia secara publik melalui platform Hugging Face dalam format Apache Arrow. Distribusi label sentimen pada dataset ini relatif seimbang antara kelas positif dan negatif, sehingga cocok untuk tugas klasifikasi biner.

Yelp Review Dataset yang digunakan dalam penelitian ini terdiri atas 30.000 ulasan pengguna terhadap berbagai jenis layanan dan bisnis, termasuk restoran dan usaha ritel. Dataset ini tersedia secara publik melalui platform Hugging Face (X. Zhang, J. Zhao, dan LeCun, 2015b) dalam format Apache Arrow dengan distribusi label sentimen yang seimbang. Jumlah data dipilih agar sebanding dengan dataset IMDB yang digunakan pada penelitian ini.

Kedua dataset memiliki label sentimen biner, dengan nilai 1 untuk sentimen positif dan nilai 0 untuk sentimen negatif. Beberapa cuplikan ulasan dari masing-masing dataset ditampilkan pada Tabel 4.2 dan Tabel 4.3.

Tabel 4.2: Cuplikan data ulasan IMDB (Maas *et al.*, 2011b).

No	Text	Label
1	An absolute masterpiece. The storytelling is compelling from start to finish, and the performances are nothing short of extraordinary. Highly recommended to anyone who appreciates great cinema.	1
2	A complete waste of time. The plot makes no sense, the acting is wooden, and the ending is thoroughly unsatisfying. Save yourself two hours and skip this one.	0
⋮	⋮	⋮
50.000	The film captures a quiet kind of grief beautifully. Not much happens on the surface, but every scene carries emotional weight. A slow burn that rewards patient viewers.	1

Tabel 4.3: Cuplikan data ulasan Yelp (X. Zhang, J. Zhao, dan LeCun, 2015b).

No	Text	Label
1	Hands down the best ramen I have had outside of Japan. The broth is rich and complex, the noodles perfectly chewy. The staff were attentive without being intrusive. Will absolutely return.	1
2	Ordered delivery and the food arrived nearly an hour late and completely cold. When I called to complain, the staff were dismissive. Would not order from here again.	0
⋮	⋮	⋮
30.000	Decent enough for a quick lunch but nothing memorable. The portions are on the smaller side for the price, and the ambiance is a bit too noisy to hold a conversation comfortably.	0

4.3.2 Prapengolahan Data

Tahap prapengolahan bertujuan membersihkan teks dari elemen yang tidak mengandung informasi bahasa sebelum diproses oleh BERT. Karena BERT merupakan model berbasis *transformer* yang menangkap hubungan kontekstual antar-token, penelitian ini menerapkan prapengolahan yang minimalis. Tahapan seperti *stemming*, penghapusan *stop words*, dan normalisasi tanda baca tidak diterapkan, karena kata-kata fungsional seperti *stop words* masih mengandung informasi kontekstual yang relevan bagi model.

Selain itu, BERT menggunakan tokenisasi berbasis WordPiece yang mampu memecah kata di luar kosakata menjadi unit *subword*, sehingga normalisasi kata secara manual tidak diperlukan. Prapengolahan pada penelitian ini difokuskan

pada pembersihan elemen non-linguistik tanpa menghilangkan kandungan informasi dalam teks. Tahapan yang diterapkan adalah sebagai berikut.

1. *Casefolding*, yaitu mengubah seluruh teks menjadi huruf kecil untuk menghindari perbedaan representasi akibat variasi kapitalisasi.
2. Penghapusan tag HTML, yaitu menghapus elemen HTML seperti `<br />` dan `<b>` yang tidak memiliki kandungan bahasa.
3. Penghapusan URL, yaitu menghapus tautan dengan pola `http://...` atau `www....` karena tidak membawa informasi sentimen maupun topik.
4. Penghapusan karakter non-ASCII, yaitu menghapus emoji, simbol unicode, dan karakter khusus yang dapat mengganggu proses tokenisasi.
5. Normalisasi *whitespace*, yaitu menyeragamkan spasi berlebih, karakter tab, dan baris baru menjadi satu spasi tunggal.

Hasil dari tahap ini adalah teks yang bersih dari elemen non-linguistik namun tetap mempertahankan struktur kalimat aslinya. Perbandingan teks sebelum dan sesudah prapengolahan dapat dilihat pada Tabel 4.4.

Tabel 4.4: Contoh hasil prapengolahan pada dataset IMDB dan Yelp.

Sebelum Prapengolahan	Sesudah Prapengolahan
<i>IMDB</i>	
An absolute MASTERPIECE. The storytelling is compelling. <code>
</code> Highly recommended to anyone who appreciates great cinema!	an absolute masterpiece. the storytelling is compelling. highly recommended to anyone who appreciates great cinema!
A complete waste of time. <code>
</code> Read the full review at http://www.filmcritic.org/reviews/2024	a complete waste of time. read the full review at
<i>Yelp</i>	
Hands down the BEST ramen outside of Japan :) The broth is incredible! Check the menu at www.ramenhouse.com	hands down the best ramen outside of japan the broth is incredible! check the menu at
Ordered delivery & the food arrived cold after 1hr. Staff were rude when I called :(Would NOT recommend.	ordered delivery the food arrived cold after 1hr. staff were rude when i called would not recommend.

Tabel 4.4 menunjukkan bahwa perubahan yang dilakukan terbatas pada penghapusan elemen non-linguistik dan penyeragaman format teks, bukan penyederhanaan isi kalimat. Dengan demikian, kandungan kontekstual dalam ulasan tetap terjaga untuk tahap ekstraksi fitur menggunakan BERT.

4.3.3 Representasi Teks

Setelah prapengolahan, teks diubah menjadi representasi numerik melalui dua tahap, yaitu tokenisasi dan ekstraksi fitur menggunakan BERT. Representasi yang dihasilkan menjadi masukan bagi *autoencoder* pada tahap pelatihan berikutnya.

Tokenisasi dilakukan menggunakan tokenizer WordPiece dari model BERT *pre-trained*. Setiap teks dikonversi menjadi urutan *token ID* dengan panjang maksimum $t = 128$ token. Teks yang melebihi batas ini akan dipotong (*truncation*), sedangkan teks yang lebih pendek akan ditambahkan token [PAD] bernilai 0 agar panjang urutan seragam. Contoh hasil tokenisasi ditampilkan pada Tabel 4.5.

Tabel 4.5: Contoh proses tokenisasi menggunakan WordPiece BERT.

Teks (hasil prapengolahan)	Hasil tokenisasi (<i>token ID</i>)
an absolute masterpiece the storytelling is compelling	[101, 2019, 7927, 4902, 1996, 4129, 2003, 17075, 102, 0, ..., 0]

Urutan *token ID* tersebut selanjutnya diproses oleh BERT untuk menghasilkan representasi kontekstual. Model yang digunakan adalah BERT *pre-trained* dengan *hidden size* 768 dan bobot yang dibekukan sepenuhnya (*frozen*). Dari keluaran BERT berupa matriks berukuran $t \times 768$, diambil baris pertama yang bersesuaian dengan token [CLS] sebagai representasi dokumen:

$$\mathbf{f} = \mathbf{h}_1 \in \mathbb{R}^{768} \quad (4.1)$$

dengan $\mathbf{h}_1$ adalah vektor keluaran BERT pada posisi token [CLS]. Vektor $\mathbf{f}$ ini merepresentasikan konteks global dokumen dan digunakan sebagai masukan bagi *autoencoder* pada tahap berikutnya. Contoh ilustrasi perubahan representasi dari *token ID* ke vektor BERT ditampilkan pada Tabel 4.6.

Tabel 4.6: Contoh representasi vektor [CLS] hasil ekstraksi BERT.

<i>Token ID</i>	Representasi [CLS] $\mathbf{f} \in \mathbb{R}^{768}$
[101, 2019, 7927, 4902, ...]	[0.31, -0.17, 0.08, ..., -0.22]

4.3.4 Pembagian Data

Setelah prapengolahan, data dari kedua dataset dibagi menjadi data latih (*training set*) dan data uji (*test set*) dengan proporsi 80% untuk data latih dan 20% untuk

data uji, mengacu pada skema yang digunakan oleh (Gowandi, Murfi, dan Nurrohmah, 2021). Pembagian ini dilakukan agar proses pelatihan dan evaluasi model dapat dilakukan secara terpisah.

Pada tahap pra-pelatihan *autoencoder*, 10% dari data latih dialokasikan sebagai data validasi untuk memantau proses pelatihan melalui nilai *validation loss*. Data validasi tidak digunakan untuk memperbarui parameter model maupun sebagai data evaluasi akhir, sedangkan data uji hanya digunakan untuk mengevaluasi performa model setelah proses pelatihan selesai.

Data latih digunakan sebagai masukan dalam proses optimasi parameter model, baik pada komponen analisis sentimen maupun pendeteksian topik. Data uji digunakan untuk mengukur performa model pada data yang belum pernah dilihat selama pelatihan.

4.4 Implementasi Model

Implementasi pada penelitian ini mencakup tiga model, yaitu JDST-DEC sebagai model yang diusulkan, NN-BERT sebagai model pembanding untuk analisis sentimen, dan BERTopic sebagai model pembanding untuk pendeteksian topik. JDST-DEC dilatih melalui dua tahap, yaitu pra-pelatihan *autoencoder* dan *joint training*. Seluruh model diimplementasikan menggunakan Python dengan pustaka sebagaimana tercantum pada Tabel 4.1.

4.4.1 Konfigurasi JDST-DEC

Konfigurasi JDST-DEC mencakup parameter pada tahap pra-pelatihan *autoencoder* dan pembentukan kluster menggunakan DEC. Konfigurasi masing-masing komponen disajikan pada Tabel 4.7 dan Tabel 4.8.

Tabel 4.7: Konfigurasi pra-pelatihan *autoencoder*

Parameter	Nilai
Arsitektur	FC, simetris
Dimensi lapisan	$768 \rightarrow 2022 \rightarrow 2022 \rightarrow 256$
Dimensi laten	256
Fungsi aktivasi	ReLU
Fungsi <i>loss</i>	MSE
<i>Optimizer</i>	Adam
<i>Learning rate</i>	2×10^{-3}
<i>Batch size</i>	128
Epoch	30

Tabel 4.8: Konfigurasi komponen DEC

Parameter	Nilai
Lapisan klaster	1
Dimensi ruang klaster	256
Jumlah klaster (K)	30, 50, 80
<i>Soft assignment</i>	Student- t
Derajat kebebasan (v)	1
<i>Loss clustering</i>	KL Divergence
Inisialisasi centroid	k -means

Dimensi masukan sebesar 768 mengikuti ukuran vektor [CLS] yang dihasilkan oleh BERT. Representasi tersebut kemudian direduksi ke ruang laten berdimensi 256 untuk menghasilkan fitur yang lebih ringkas sebelum proses klasterisasi. Pada tahap DEC, centroid klaster diinisialisasi menggunakan metode k -means, sedangkan jumlah klaster (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 klaster, guna menganalisis pengaruh jumlah topik terhadap kualitas hasil klasterisasi.

4.4.2 Konfigurasi *Joint Training*

Setelah tahap pra-pelatihan, JDST-DEC dioptimasi melalui proses *joint training* untuk menjalankan dua tugas secara simultan, yaitu klasifikasi sentimen dan pembentukan klaster topik. Fungsi objektif yang digunakan didefinisikan sebagai

$$\mathcal{L} = \eta \mathcal{L}_{\text{sent}} + \gamma \mathcal{L}_{\text{clust}}.$$

Konfigurasi pelatihan yang digunakan pada tahap *joint training* ditunjukkan pada Tabel 4.9.

Tabel 4.9: Konfigurasi *joint training* JDST-DEC

Parameter	Nilai
<i>Learning rate</i>	1×10^{-3}
<i>Batch size</i>	256
Iterasi maksimum	100
<i>Update interval</i>	10
Toleransi (τ)	10^{-4}
Bobot sentimen (η)	0,5
Bobot klaster (γ)	0,5
<i>Optimizer</i>	SGD
Jumlah topik (K)	30, 50, dan 80
<i>Seed</i>	42

Nilai η dan γ ditetapkan sebesar 0,5 untuk memberikan kontribusi yang seimbang antara tugas klasifikasi sentimen dan klasterisasi topik selama proses pelatihan. Sementara itu, jumlah topik (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 topik, untuk mengidentifikasi konfigurasi yang memberikan hasil terbaik.

4.4.3 Konfigurasi Model Pembandingan

Penelitian ini menggunakan dua model pembandingan untuk mengevaluasi kinerja JDST-DEC pada masing-masing tugas. NN-BERT digunakan sebagai model pembandingan untuk analisis sentimen karena memanfaatkan representasi BERT yang sama tanpa mekanisme pembelajaran multitugas. Sementara itu, BERTopic digunakan sebagai pembandingan pada pendeteksian topik karena merupakan salah satu pendekatan berbasis *transformer* yang banyak digunakan untuk *topic modeling*.

4.4.3.1 NN-BERT

NN-BERT digunakan sebagai model dasar (*baseline*) untuk tugas analisis sentimen. Representasi [CLS] yang dihasilkan oleh BERT digunakan sebagai masukan bagi lapisan klasifikasi *fully connected* untuk memprediksi kelas sentimen. Konfigurasi model NN-BERT ditunjukkan pada Tabel 4.10.

Tabel 4.10: Konfigurasi model NN-BERT

Parameter	Nilai
<i>Encoder</i>	BERT, <i>frozen</i>
<i>Dimensi input</i>	768 ([CLS])
<i>Classifier</i>	FC, 1 lapisan
<i>Dimensi tersembunyi</i>	256
<i>Fungsi aktivasi</i>	ReLU
<i>Dimensi output</i>	2 kelas
<i>Fungsi loss</i>	<i>Cross-Entropy</i>
<i>Optimizer</i>	Adam
<i>Learning rate</i>	5×10^{-5}
<i>Batch size</i>	256
<i>Epoch</i>	10

4.4.3.2 BERTopic

BERTopic digunakan sebagai model pembandingan pada tugas pendeteksian topik. Model ini memanfaatkan representasi semantik berbasis BERT untuk memperoleh representasi dokumen, kemudian dilanjutkan dengan reduksi dimensi menggunakan UMAP sebelum pembentukan topik dilakukan melalui algoritma klusterisasi. Pada penelitian ini, BERTopic diimplementasikan menggunakan dua algoritma klusterisasi, yaitu HDBSCAN dan K-Means, untuk mengevaluasi pengaruh metode pembentukan kluster terhadap kualitas topik yang dihasilkan. Konfigurasi model BERTopic ditunjukkan pada Tabel 4.11.

Tabel 4.11: Konfigurasi model BERTopic

Komponen	Parameter	Nilai
<i>Embedding</i>	Model	bert-base-uncased
	Dimensi	768
UMAP	n_neighbors	15
	n_components	5
	min_dist	0,0
	Metrik	Kosinus
HDBSCAN	min_cluster_size	10
	min_samples	5
	Metrik	Euclidean
	prediction_data	True
K-Means	n_clusters	30, 50, 80
Topik	Jumlah topik (K)	30, 50, 80
	Metode reduksi	nr_topics
	Representasi topik	c-TF-IDF

Untuk menjaga konsistensi dengan JDST-DEC, jumlah topik yang dievaluasi pada BERTopic ditetapkan sebesar 30, 50, dan 80 topik. Pengaturan ini dilakukan agar perbandingan antar metode dilakukan pada ruang solusi yang setara, sehingga perbedaan hasil yang diperoleh lebih merefleksikan kemampuan model dalam membentuk representasi topik dibandingkan pengaruh jumlah topik yang digunakan.

4.4.3.3 MTL-GUI

MTL-GUI digunakan sebagai model pembanding utama pada tugas analisis sentimen dan pendeteksian topik secara simultan (Gui *et al.*, 2022). Model ini menggunakan *Hierarchical Attention Network* (HAN) berbasis GRU sebagai *shared encoder* untuk membangun representasi dokumen bersama, yang kemudian digunakan secara langsung untuk klasifikasi sentimen. Untuk pendeteksian topik, representasi yang sama diproses lebih lanjut menggunakan *Variational Autoencoder* (VAE) sebelum dikelompokkan menggunakan K-Means. Kedua tugas dihubungkan melalui mekanisme *attention sharing* yang mendorong konsistensi antara bobot atensi kata pada HAN dan distribusi kata pada model topik.

Konfigurasi komponen HAN ditunjukkan pada Tabel 4.12, konfigurasi VAE pada Tabel 4.13, dan konfigurasi pelatihan pada Tabel 4.14.

Tabel 4.12: Konfigurasi komponen HAN pada MTL-GUI

Parameter	Nilai
Dimensi <i>embedding</i>	200
<i>Word-level</i> GRU	Bidireksional, 2 lapisan
Ukuran <i>hidden word</i> GRU	150 (300 setelah konkatenasi)
<i>Sent-level</i> GRU	Bidireksional, 1 lapisan
Ukuran <i>hidden sent</i> GRU	150 (300 setelah konkatenasi)
<i>Classifier</i>	FC, 2 lapisan
Dimensi tersembunyi <i>classifier</i>	100
Fungsi aktivasi	LeakyReLU, Tanh
<i>Dropout</i>	0,3
Dimensi <i>output</i>	2 kelas

Tabel 4.13: Konfigurasi komponen VAE pada MTL-GUI

Parameter	Nilai
Dimensi <i>input</i> (BoW)	5.000
Dimensi <i>hidden encoder</i>	500
Jumlah topik (K)	30, 50, 80
Lapisan <i>generator</i>	4 lapisan (tanh)
Fungsi aktivasi <i>output</i>	Softmax
<i>Dropout</i>	0,2
Fungsi <i>loss</i> topik	ELBO (NLL + KLD)

Komponen VAE pada MTL-GUI dilatih dengan memaksimalkan *Evidence Lower Bound* (ELBO), yaitu batas bawah dari *log-likelihood* data yang umum digunakan pada VAE karena *likelihood* sebenarnya tidak dapat dihitung secara langsung (Kingma dan Welling, 2013):

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) \parallel p(z)).$$

Suku pertama merupakan *Negative Log-Likelihood* (NLL) yang mengukur kualitas rekonstruksi dokumen dari ruang laten z , sedangkan suku kedua adalah KL Divergence yang menjaga distribusi laten $q(z|x)$ tetap mendekati distribusi *prior* $p(z)$. Fungsi *loss* yang dioptimalkan merupakan negasi dari ELBO, sehingga pada Tabel 4.13 dituliskan sebagai penjumlahan NLL dan KLD.

Tabel 4.14: Konfigurasi pelatihan MTL-GUI

Parameter	Nilai
Ukuran kosakata	12.000
Ukuran kosakata topik	5.000
Fungsi <i>loss</i> sentimen	<i>Cross-Entropy</i>
Fungsi <i>loss attention sharing</i>	KL Divergence
Bobot <i>loss</i> sentimen (w_c)	9,0
Bobot <i>loss</i> topik (w_t)	1,0
Interval <i>attention sharing</i>	Setiap 50 langkah
<i>Optimizer</i>	Adam
<i>Learning rate</i> utama	1×10^{-3}
<i>Learning rate attention</i>	1×10^{-4}
<i>Weight decay</i>	1×10^{-8}
<i>Batch size</i>	32
Epoch	50

Secara keseluruhan, konfigurasi ini digunakan sebagai pengaturan standar dalam eksperimen MTL-GUI.

4.5 Analisis Kinerja Model

Bagian ini menyajikan hasil evaluasi model pada dua tahapan utama, yaitu pra-pelatihan (*pretraining*) *autoencoder* dan proses *joint training*. Evaluasi dilakukan pada dataset IMDB dan Yelp untuk melihat konsistensi performa model pada domain ulasan yang berbeda. Tahap pra-pelatihan digunakan untuk mengevaluasi kemampuan model dalam mempelajari representasi laten sebelum proses pembelajaran multitugas, sedangkan tahap *joint training* digunakan untuk menilai performa akhir model pada tugas analisis sentimen dan pendeteksian topik.

4.5.1 Analisis Kinerja Pra-pelatihan

Tahap pra-pelatihan dilakukan untuk membentuk representasi laten awal melalui mekanisme rekonstruksi pada *autoencoder*. Representasi ini selanjutnya digunakan sebagai inisialisasi sebelum proses *joint training*, sehingga kualitas hasil rekonstruksi menjadi penting karena dapat memengaruhi proses pembelajaran pada tahap berikutnya.

Evaluasi kinerja pra-pelatihan dilakukan menggunakan *Mean Squared Error* (MSE) pada data latih dan data validasi. Nilai MSE digunakan untuk mengukur

selisih antara representasi masukan dan hasil rekonstruksi yang dihasilkan model, di mana nilai yang lebih kecil menunjukkan kemampuan rekonstruksi yang lebih baik. Perbandingan antara MSE data latih dan data validasi juga digunakan untuk mengamati kestabilan proses pelatihan serta indikasi terjadinya *overfitting*.

Hasil pra-pelatihan pada dataset IMDB dan Yelp disajikan pada Tabel 4.15 dan Tabel 4.16.

Tabel 4.15: Perkembangan *loss* pra-pelatihan pada dataset IMDB

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	$2,370 \times 10^{-3}$	$2,544 \times 10^{-3}$
5	$3,300 \times 10^{-4}$	$3,610 \times 10^{-4}$
10	$3,780 \times 10^{-4}$	$4,120 \times 10^{-4}$
15	$1,570 \times 10^{-4}$	$1,760 \times 10^{-4}$
20	$1,370 \times 10^{-4}$	$1,550 \times 10^{-4}$
25	$1,440 \times 10^{-4}$	$1,620 \times 10^{-4}$
30	$1,410 \times 10^{-4}$	$1,590 \times 10^{-4}$

Pra-pelatihan *autoencoder* pada dataset IMDB menunjukkan penurunan MSE yang signifikan dari epoch pertama hingga epoch ke-20, kemudian cenderung stabil pada kisaran $1,4 \times 10^{-4}$. *Validation loss* konsisten berada sedikit di atas *train loss*, mengindikasikan bahwa model tidak mengalami *overfitting*.

Tabel 4.16: Perkembangan *loss* pra-pelatihan pada dataset Yelp

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	$3,790 \times 10^{-3}$	$4,062 \times 10^{-3}$
5	$4,180 \times 10^{-4}$	$4,540 \times 10^{-4}$
10	$2,780 \times 10^{-4}$	$3,040 \times 10^{-4}$
15	$1,990 \times 10^{-4}$	$2,200 \times 10^{-4}$
20	$1,670 \times 10^{-4}$	$1,860 \times 10^{-4}$
25	$1,830 \times 10^{-4}$	$2,030 \times 10^{-4}$
30	$1,550 \times 10^{-4}$	$1,730 \times 10^{-4}$

Pola serupa terjadi pada dataset Yelp, dengan MSE turun tajam pada epoch-epoch awal dan mulai konvergen setelah epoch ke-20. Nilai akhir *train loss* sebesar $1,550 \times 10^{-4}$ menunjukkan bahwa *autoencoder* mampu merekonstruksi representasi BERT dengan baik pada kedua dataset.

4.5.2 Analisis Kinerja *Joint Training*

Kinerja *joint training* dievaluasi pada dua aspek utama, yaitu klasifikasi sentimen dan pemodelan topik. Evaluasi membandingkan JDST-DEC dengan beberapa model pembanding, yaitu NN-BERT, MTL-GUI (Gui *et al.*, 2022), BERTopic-HDBSCAN, dan BERTopic-KMeans.

4.5.2.1 Klasifikasi Sentimen

Performa klasifikasi sentimen dievaluasi menggunakan *accuracy* dan F1-score. Model yang tidak memiliki komponen klasifikasi sentimen tidak disertakan dalam evaluasi.

Tabel 4.17: Perbandingan hasil *Sentiment Classification*

Method	IMDB						Yelp					
	Acc			F1			Acc			F1		
NN-BERT	0.8581			0.8612			0.8664			0.8715		
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
MTL-GUI	0.8327	0.8326	0.8317	0.8317	0.8334	0.8334	0.8385	0.8385	0.8380	0.8380	0.8402	0.8401
JDST-DEC	0.8957	0.8963	0.8967	0.8986	0.8941	0.8930	0.8965	0.8944	0.8983	0.9020	0.8935	0.8953

Berdasarkan Tabel 4.17, JDST-DEC memperoleh nilai *accuracy* dan F1-score tertinggi di antara model yang dievaluasi pada pengaturan eksperimen yang sama. Pada dataset IMDB, JDST-DEC memperoleh F1-score terbaik sebesar 0.8986 pada $K = 50$, lebih tinggi dibandingkan NN-BERT (0.8612) dan MTL-GUI (0.8317). Sementara itu, pada dataset Yelp, JDST-DEC mencapai F1-score sebesar 0.9020 pada $K = 50$, melampaui NN-BERT (0.8715) dan MTL-GUI (0.8401).

Selain itu, perubahan jumlah topik dari $K = 30$ hingga $K = 80$ hanya memberikan perbedaan performa yang relatif kecil. Hal ini menunjukkan bahwa kinerja JDST-DEC tetap stabil terhadap variasi jumlah topik, baik pada dataset IMDB maupun Yelp.

4.5.2.2 Pemodelan Topik

Kualitas pemodelan topik dievaluasi menggunakan dua metrik, yaitu *Topic Coherence* (TC-NPMI) dan *Topic Diversity* (TD). TC-NPMI mengukur seberapa bermakna hubungan semantik antar kata dalam satu topik berdasarkan ko-munculan pada korpus, sedangkan TD mengukur proporsi kata unik yang muncul lintas topik. Nilai yang lebih tinggi pada kedua metrik menunjukkan topik yang

lebih koheren sekaligus lebih beragam. NN-BERT tidak diikutsertakan karena tidak memiliki komponen pemodelan topik.

Tabel 4.18: Perbandingan hasil *Topic Modelling*

Method	IMDB						Yelp					
	$K = 30$		$K = 50$		$K = 80$		$K = 30$		$K = 50$		$K = 80$	
	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD	TC-NPMI	TD
BERTopic-HDBSCAN	0.1714	0.7966	0.1823	0.7680	0.1751	0.7625	0.1092	0.7233	0.1666	0.7041	0.1646	0.6671
BERTopic-KMeans	0.1566	0.9567	0.1596	0.9000	0.1874	0.8413	0.1077	0.7800	0.1120	0.6740	0.1465	0.6438
MTL-GUI	0.0200	0.5833	0.0108	0.5800	0.0060	0.4913	0.1578	0.6600	0.1086	0.6720	0.1123	0.5000
JDST-DEC	0.0803	0.2256	0.0551	0.2100	0.0675	0.2875	0.0788	0.3267	0.0369	0.2280	0.0563	0.3150

Berdasarkan Tabel 4.18, BERTopic memperoleh nilai *Topic Diversity* (TD) tertinggi pada sebagian besar konfigurasi. Pada dataset IMDB, BERTopic-KMeans menghasilkan TD tertinggi sebesar 0.9567 ($K = 30$), 0.9000 ($K = 50$), dan 0.8413 ($K = 80$). Pada dataset Yelp, BERTopic-KMeans memperoleh TD tertinggi pada $K = 30$ sebesar 0.7800, sedangkan BERTopic-HDBSCAN unggul pada $K = 50$ dan $K = 80$ dengan nilai masing-masing 0.7041 dan 0.6671.

Untuk metrik TC-NPMI, BERTopic-HDBSCAN memperoleh nilai tertinggi pada IMDB sebesar 0.1714 ($K = 30$) dan 0.1823 ($K = 50$), sedangkan BERTopic-KMeans menghasilkan nilai tertinggi pada IMDB $K = 80$ sebesar 0.1874. Pada dataset Yelp, BERTopic-HDBSCAN memperoleh TC-NPMI tertinggi pada $K = 30$ (0.1092) dan $K = 50$ (0.1666), sedangkan BERTopic-KMeans unggul pada $K = 80$ dengan nilai 0.1465.

4.5.3 Analisis Multiloss

Proses *joint training* pada JDST-DEC mengoptimasi dua komponen *loss* secara simultan, yaitu *clustering loss* berbasis KL Divergence ($\mathcal{L}_{\text{clust}}$) dan *sentiment loss* berbasis *cross-entropy* ($\mathcal{L}_{\text{sent}}$), dengan fungsi objektif:

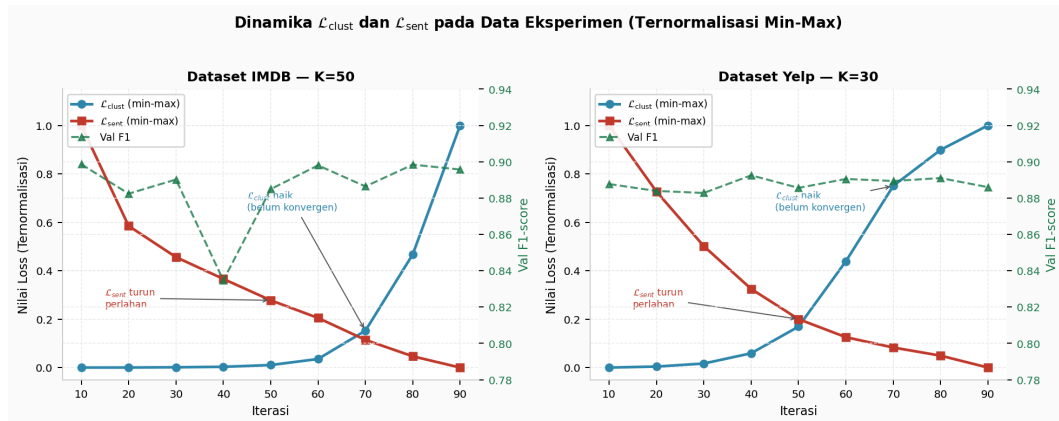
$$\mathcal{L} = \gamma \mathcal{L}_{\text{clust}} + \eta \mathcal{L}_{\text{sent}}$$

Karena kedua komponen memiliki rentang nilai yang berbeda, $\mathcal{L}_{\text{sent}}$ cenderung memiliki nilai lebih besar dibandingkan $\mathcal{L}_{\text{clust}}$, sehingga analisis dilakukan menggunakan normalisasi *min-max*. Normalisasi ini digunakan agar perubahan kedua komponen dapat diamati pada skala yang sama.

4.5.3.1 Dinamika pada Data Eksperimen

Gambar 4.2 menunjukkan perkembangan $\mathcal{L}_{\text{clust}}$ dan $\mathcal{L}_{\text{sent}}$ selama proses *joint training* pada salah satu contoh eksperimen yang dijalankan, yaitu IMDB ($K = 50$)

dan Yelp ($K = 30$).



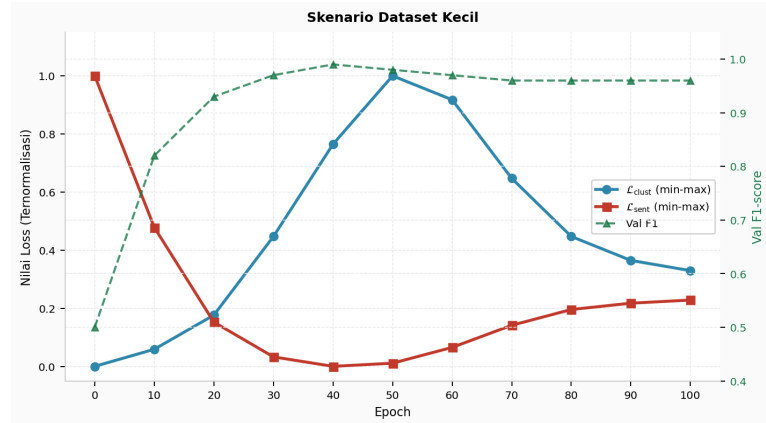
Gambar 4.2: Dinamika $\mathcal{L}_{\text{clust}}$ dan $\mathcal{L}_{\text{sent}}$ (ternormalisasi *min-max*) selama *joint training* pada dataset IMDB ($K = 50$) dan Yelp ($K = 30$).

Berdasarkan Gambar 4.2, $\mathcal{L}_{\text{sent}}$ menunjukkan tren menurun sejak iterasi awal. Sebaliknya, $\mathcal{L}_{\text{clust}}$ cenderung meningkat selama proses pelatihan dan belum menunjukkan kecenderungan konvergen hingga iterasi ke-100.

Pola tersebut dapat dikaitkan dengan mekanisme DEC. Pada tahap awal, distribusi *soft assignment* (q) masih mendekati distribusi target (p), sehingga nilai KL Divergence relatif kecil. Seiring proses pelatihan, perubahan distribusi target menyebabkan nilai KL Divergence meningkat. Hasil ini mengindikasikan bahwa proses pembentukan kluster masih berlangsung.

4.5.3.2 Eksperimen Dataset Kecil

Untuk memberikan ilustrasi tambahan mengenai dinamika *multiloss*, dilakukan percobaan pada dataset dengan ukuran lebih kecil (1000 data). Hasilnya ditunjukkan pada Gambar 4.3.



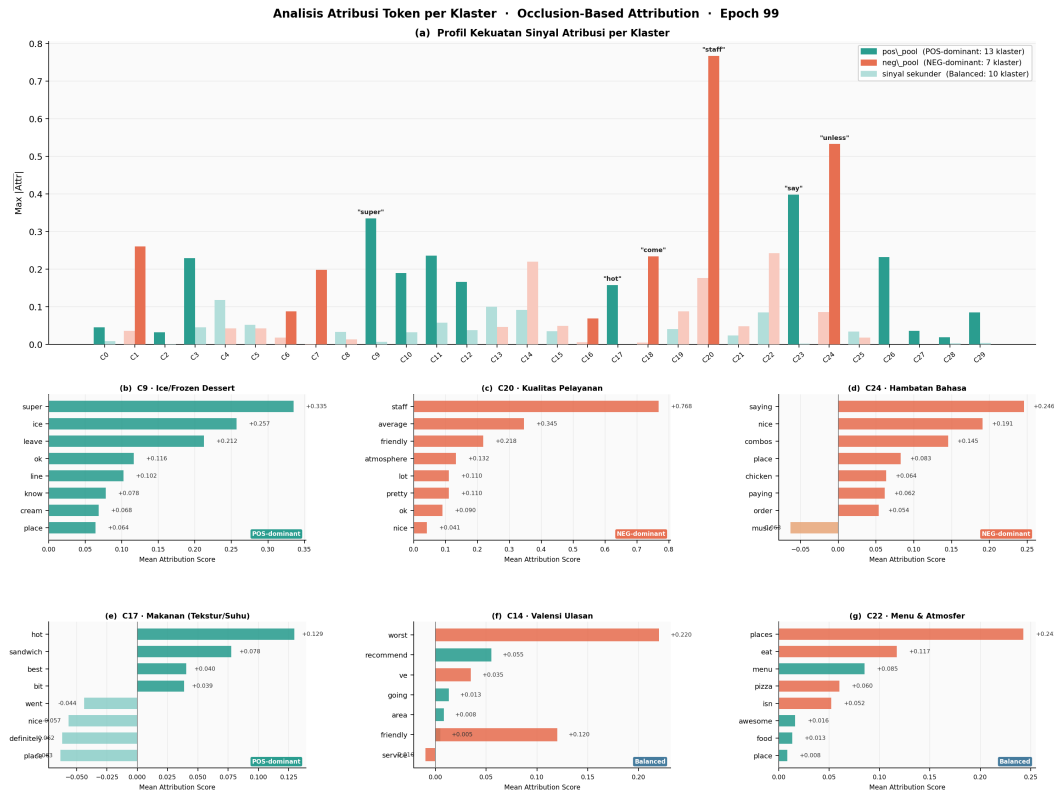
Gambar 4.3: Skenario dataset kecil: $\mathcal{L}_{\text{sent}}$ mencapai kondisi stabil lebih dahulu, kemudian $\mathcal{L}_{\text{clust}}$ mulai menurun dan kedua komponen menunjukkan pola yang lebih seimbang.

Pada Gambar 4.3, penurunan $\mathcal{L}_{\text{sent}}$ terjadi lebih cepat dibandingkan $\mathcal{L}_{\text{clust}}$. Setelah laju penurunan $\mathcal{L}_{\text{sent}}$ mulai melambat, perubahan pada komponen klusterisasi menjadi lebih terlihat dan $\mathcal{L}_{\text{clust}}$ mulai mengalami penurunan. Pola ini menunjukkan bahwa ketika proses klasifikasi sentimen telah cukup terbentuk, optimasi mulai diikuti oleh perbaikan pada struktur kluster.

4.6 Analisis Interpretabilitas Model

Analisis interpretabilitas dilakukan menggunakan metode *occlusion-based token attribution* sebagaimana dirumuskan pada Persamaan 3.26–3.29. Evaluasi dilakukan menggunakan model hasil *joint training* pada epoch terakhir (epoch ke-99), bukan model dengan performa evaluasi terbaik. Pemilihan epoch terakhir bertujuan untuk mengamati representasi dan pola atribusi yang terbentuk setelah seluruh proses pelatihan selesai.

Analisis dilakukan pada konfigurasi jumlah kluster sebesar 30 ($K = 30$) menggunakan dataset Yelp. Konfigurasi ini dipilih dari beberapa percobaan jumlah kluster yang digunakan pada penelitian, yaitu $K = 30$, $K = 50$, dan $K = 80$. Analisis interpretabilitas dibatasi pada konfigurasi ini sebagai studi kasus untuk memberikan gambaran perilaku model secara kualitatif. Untuk setiap kluster, dihitung skor atribusi rata-rata token terhadap masing-masing kelas sentimen, yaitu *pos_pool* dan *neg_pool*. Gambar 4.4 memperlihatkan ringkasan atribusi token pada seluruh 30 kluster.



Gambar 4.4: Ringkasan hasil atribusi token pada 30 kluster. Panel (a) menunjukkan nilai atribusi tertinggi pada setiap kluster, sedangkan panel (b)–(g) menampilkan token dengan kontribusi terbesar pada beberapa kluster terpilih berdasarkan skor atribusi rata-rata.

Pada kelompok kluster dengan dominansi positif, beberapa token yang sering muncul sebagai kontributor utama antara lain “*super*”, “*hot*”, “*sandwich*”, dan “*amazing*”. Token-token tersebut menunjukkan bahwa model cenderung memanfaatkan kata yang bersifat evaluatif serta kata yang berkaitan dengan konteks makanan dalam menghasilkan prediksi positif. Namun, temuan ini hanya menunjukkan pola kontribusi token dan belum cukup untuk digunakan sebagai dasar dalam menentukan tema kluster secara pasti.

Pada kluster dengan dominansi negatif, token seperti “*staff*”, “*average*”, dan “*atmosphere*” memiliki kontribusi yang cukup besar terhadap prediksi negatif. Selain itu, ditemukan beberapa token dengan skor negatif pada *neg_pool*, seperti “*unless*” dan “*english*”, yang menunjukkan bahwa pengaruh token terhadap prediksi model tidak selalu sesuai dengan polaritas kata secara umum. Hal ini menunjukkan bahwa model juga mempertimbangkan konteks kemunculan token, bukan hanya keberadaan kata secara terpisah.

Pada kluster dengan distribusi sentimen yang lebih seimbang, ditemukan token positif dan negatif yang muncul secara bersamaan, seperti “*recommend*” dan “*worst*”. Kondisi ini menunjukkan bahwa beberapa kluster masih memiliki variasi

sentimen yang cukup tinggi dan belum sepenuhnya terpisah berdasarkan kecenderungan sentimennya.

Secara keseluruhan, hasil atribusi menunjukkan bahwa model telah menangkap pola penggunaan kata yang relevan terhadap tugas analisis sentimen. Namun, interpretasi pada tingkat klaster masih memiliki keterbatasan karena kontribusi token yang ditemukan belum selalu menghasilkan pemisahan topik maupun sentimen secara konsisten.